Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Гантөмөрийн Алтай

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Бакалаврын төгсөлтийн ажил

Улаанбаатар хот

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Компьютерын ухааны тэнхим

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Мэргэжлийн индекс: 071405000000002306
Мэргэжил: Программ хангамжийн инженерчлэл

Удирдагч: Магистр (M.Sc) Э.Батцэцэг
Зөвлөгч: Доктор (Ph.D), Дэд профессор Г.Ганбат
Гүйцэтгэгч: Г.Алтай

Улаанбаатар хот
2026 он 6 сар

Батлав. Компьютерын ухааны тэнхимын эрхлэгч:

..... /Доктор (Ph.D), дэд профессор А.Хүдэр/

Удирдагч:

..... /Магистр (M.Sc) Э.Батцэцэг/

ТӨГСӨЛТИЙН АЖЛЫН ТӨЛӨВЛӨГӨӨ

Сэдэв: "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт"

№	Ажлын бүлэг, хэсгийн нэр	Эзлэх хувь	Дуусах хугацаа
1	Удиртгал хэсэг	5%	2026-2-4
2	Судалгааны хэсэг	15%	2026-2-21
3	Шинжилгээний хэсэг	15%	2026-3-4
4	Зохиомжийн хэсэг	20%	2026-4-1
5	Хөгжүүлэлтийн хэсэг	40%	2026-4-29
6	Дүгнэлт	5%	2026-5-27

Төлөвлөгөөг боловсруулсан оюутан: ..... /Г.Алтай/

ТӨГСӨЛТИЙН АЖЛЫН ЯВЦ

№	Хийж гүйцэтгэсэн ажил	Биелсэн хугацаа	Удирдагчийн гарын үсэг
1	Удиртгал	2026-2-4	
2	Судалгааны хэсэг	2026-2-21	
3	Шинжилгээний хэсэг	2026-3-4	
4	Зохиомжийн хэсэг	2026-4-1	
5	Хөгжүүлэлтийн хэсэг	2026-4-29	
6	Дүгнэлт	2026-5-27	

Ажлын товч дүгнэлт

.....

.....

.....

.....

.....

.....

.....

Удирдагч: ..... /Магистр (M.Sc) Э.Батцэцэг/

ЗӨВШӨӨРӨЛ

Оюутан Г.Алтай-н бичсэн төгсөлтийн ажлыг УШК-д хамгаалуулахаар тодорхойлов.

Салбарын эрхлэгч: ..... /Доктор (Ph.D), дэд профессор А.Хүдэр/

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

ШҮҮМЖИЙН ХУУДАС

Компьютерын ухааны тэнхимын төгсөх курсийн оюутан Г.Алтайгын "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт" сэдэвт төгсөлтийн ажлын шүүмж:

1. Төслөөр дэвшүүлсэн асуудал, үүнтэй холбоотой онолын материал уншиж судалсан байдал; Энэ талаар хүмүүсийн хийсэн судалгаа, түүний үр дүнг уншиж тусгасан эсэх:

   .....
   .....
   .....
   .....
   .....
   .....
   .....

2. Төслийн ерөнхий агуулга, шийдсэн зүйлс, хүрсэн үр дүн; Өөрийн санааг гарган, харьцуулалт хийн, дүгнэж байгаа чадвар:

   .....
   .....
   .....
   .....
   .....
   .....
   .....

3. Эмх цэгцтэй, стандарт хангасан өөрөөр хэлбэл диплом бичих шаардлагуудыг биелүүлсэн эсэх; Төсөлд анзаарагдсан алдаанууд, зөв бичгийн болон өгүүлбэр зүйн гэх мэт /Хуудас дугаарлагдаагүй, зураг хүснэгтийн дугаар, тайлбар байхгүй, шрифт хольсон, хувилсан зүйл ихээр оруулсан/:

.....

.....

.....

.....

.....

.....

4. Төслөөр орхигдуулсан болон дутуу болсон зүйлс; Цаашид анхаарах хэрэгтэй зүйлс:

.....

.....

.....

.....

.....

.....

5. Төслийн талаар онцолж тэмдэглэх зүйлс:

.....

.....

.....

.....

.....

.....

6. Ерөнхий оноо (5 оноо)

.....

Шүүмж бичсэн: ..... /Магистр (M.Sc) О.Нэр/

Ажлын газар: .....

Хаяг (Утас) .....

Зохиогч эрхийн хамгаалал

Миний бие Г.Алтай, "Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт" сэдэвт энэ ажил нь минийх бөгөөд дараахыг нотолж байна. Үүнд:

- Горилогч энэ ажлыг тус сургуулиас боловсролын зэрэг авахаар бүхэлд нь буюу голлон хийсэн болно.
- Энэ ажлын аль нэг хэсгийг тус сургуульд эсвэл өөр байгууллагад боловсролын зэрэг, мэргэшил авахаар өмнө нь илгээсэн бол түүнийгээ тодорхой заасан болно.
- Бусад хүмүүсийн хэвлүүлсэн ажлаас зөвлөгөө авсан бол түүнийгээ үндэслэсэн болно.
- Бусад хүмүүсийн ажлаас ишлэл хийхдээ гол эх үүсвэрийг нь заасан болно.
- Миний ажилд тусалсан голлох бүх эх үүсвэрт талархаж байна.
- Ажлыг бусадтай хамтарсан бол алийг нь бусад хүмүүс хийсэн болохыг тодорхой заасан болно.

Гарын үсэг: _____

Огноо: _____

"*Thanks to my solid academic training, today I can write hundreds of words on virtually any topic without possessing a shred of information, which is how I got a good job in journalism.*"

Dave Barry

Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль

Хураангуй

Хувийн үүлэн хадгалалтын системийн зохиомж ба хэрэгжүүлэлт

Г.Алтай
bbyyydriver@gmail.com

Түлхүүр үгс: Үүлэн хадгалалтын систем, веб систем, IOS native мобайл app

Энэхүү дипломын ажлаар хэрэглэгчийн өгөгдлийг гуравдагч талын үүлэн үйлчилгээнд бус, өөрийн хяналтын орчинд хадгалах боломжтой хувийн үүлэн хадгалалтын системийг зохион бүтээж, хэрэгжүүлэв. Ажлын зорилго нь өөрөө байршуулж ажиллуулах боломжтой, хөнгөн бүтэцтэй систем боловсруулж, хэрэглэгч веб болон iOS клиентээр дамжуулан файлуудаа удирдах нөхцөлийг бүрдүүлэхэд оршино.

Ажлын хүрээнд ижил төст үүлэн хадгалалтын системүүдийг судалж, системийн шаардлага, үндсэн хэрэглэгчийн урсгал, өгөгдлийн зохиомж, архитектурын шийдлийг тодорхойлсон. Үүний үндсэн дээр сервер талыг Rust болон PostgreSQL, веб клиентийг Angular, iOS клиентийг Swift/SwiftUI технологиор хэрэгжүүлэв. Хөгжүүлсэн систем нь анхны тохиргоо хийх, хэрэглэгч нэвтрэх, файл болон хавтас үүсгэх, байршуулах, татах, нэр өөрчлөх, устгах, хуваалцах эрх олгох, админ хэрэглэгчээр бусад хэрэглэгчийг удирдах зэрэг үндсэн боломжуудыг хангаж байна.

Туршилтын үр дүнгээр системийн үндсэн функцууд тогтвортой ажиллаж, хэрэглэгч өөрийн сервер дээр өгөгдлөө хадгалан веб болон мобайл орчноос хандах боломжтой болох нь харагдсан. Иймээс энэхүү ажил нь өгөгдлийн эзэмшил, нууцлал, зардлын хяналтыг сайжруулахад чиглэсэн self-hosted хувийн үүлэн хадгалалтын хөнгөн шийдэл боловсруулах практик үндэс болж байна.

Талархал

Энэхүү дипломын ажлыг хийж дуусгахад хамгийн гол нөлөө үзүүлсэн, сэдэв бүрийг нягталж, үнэтэй зөвөлгөө өгч, алдааг маань засаж, удирдан чиглүүлж байсан удирдагч багш Э.Батцэцэг багшдаа хамгийн түрүүнд чин сэтгэлээсээ баярлалаа гэж хэлмээр байна. Мөн дипломын ажлын зөвлөх багш болох Г.Ганбат багшдаа үнэт цагаа зарцуулж, зааж, сургаж, орхигдуулсан зүйлсийг сануулж, хэрэгтэй зөвлөгөөнүүд өгч байсанд гүн талархлаа илэрхийлье.

Намайг бакалаврын дипломын ажил бичиж чадах хэмжээнд хүртэл зааж сургасан МХТС -н Компьютерын Ухааны салбарын багш нар, “Программ хангамж” мэргэжлийн хөтөлбөрийн хүрээнд хичээл зааж байсан ШУТИС -н багш нартаа баярлалаа. Суралцахад аюулгүй, цэвэрхэн орчныг бүрдүүлж өгч байсан МХТС -н ажилчдад баярлалаа.

Академик мэдлэгээ практик дээр туршин, дипломын ажлын хүрээнд шинэ технологиудыг судлан, мэргэжлийн хувьд туршлагажих боломж олгож байсан Грэйп-сити Монгол компанийн “Технелогийн Газар” -н захирал болон ахлах инженертээ талархлаа илэрхийлмээр байна.

Эцэст нь, би дипломын ажилдаа их цаг зарцуулах боломжийг олгож, олон талаар чухал дэмжлэг үзүүлж байсан гэр бүлийнхэн болон урам зориг, эрч хүчээр урамшуулж байсан найзууддаа ч бас чин сэтгэлийн талархлаа илэрхийлэх ёстой.

Товчилсон үгс

LAN	**L**ocal **A**rea **N**etwork
E2EE	**E**nd-to-**E**nd **E**ncryption
API	**A**pplication **P**rogramming **I**nterface
CPU	**C**entral **P**rocessing **U**nit
OS	**O**perating **S**ystem
IDE	**I**ntegrated **D**evelopment **E**nvironment
SQL	**S**tructured **Q**uery **L**anguage
HTML	**H**ypertext **M**arkup **L**anguage
XML	E**x**tensible **M**arkup **L**anguage
JSON	**J**avascript **O**bject **N**otation

Гарчиг

Зохиогч эрхийн хамгаалал **i**

Хураангуй **iii**

Талархал **iv**

Товчилсон үгс **v**

Удиртгал **1**

1 Хувийн үүлэн хадгалалтын системийн тухай онол, арга зүйн судалгаа **4**
1.1 Үүлэн хадгалалтын системийн ерөнхий ойлголт 4
1.2 Ижил төст системийн судалгаа 6
1.3 Ашиглах технологиуд 17

2 Системийн шаардлага ба шинжилгээ **20**
2.1 Системийн үйл ажиллагааны дэлгэрэнгүй 20
2.2 Usecase диаграмм 21
2.3 Юзкейсийн тодорхойлолт 23
2.4 Функциональ шаардлага 44
2.5 Функциональ бус шаардлага 45
2.6 Класс диаграм 46
2.7 Дараалын диаграмм 48
2.8 Ажлын хүрээ 50

3 Хувийн үүлэн хадгалалтын системийн зохиомж ба хөгжүүлэлт **52**
3.1 Системийн архитектур 52
3.2 Зохиомжийн үндсэн компонентууд 52
3.3 Өгөгдлийн ерөнхий схем 53
3.4 Системийн прототип 54
3.5 Хөгжүүлсэн системийн интерфейс 57
3.6 Системд хийгдсэн тест 57
3.7 Системийн нэвтрүүлэлт 57

Ерөнхий дүгнэлт **59**

A Серверийн API-ийн жишээ **60**
A.1 REST endpoint-ууд 60

Ашигласан материалын жагсаалт **61**

Зургийн жагсаалт

1.1 Хэрэглэгчдийн ашиглаж буй үүлэн хадгалалтын үйлчилгээний үзүүлэлт 5
1.2 Colossus control plane-ийн ерөнхий бүтэц (эх сурвалж: Google Cloud Blog) [2] 8
1.3 Nextcloud-ийн хандалтын ба хадгалалтын давхаргын ерөнхий зураглал (эх сурвалж: Nextcloud Architecture Whitepaper) [13] 10
1.4 Docker орчинд ажиллуулсан Nextcloud-ийн веб интерфэйсийн дэлгэцийн зураг 11
1.5 Docker дээр ажиллуулсан Nextcloud AIO контейнерүүдийн санах ойн хэрэглээний дэлгэцийн зураг 11
1.6 ownCloud Infinite Scale-ийн system context зураг (эх сурвалж: ownCloud docs) [23] 13

2.1 Usecase диаграм 22
2.2 Хувийн үүлэн хадгалалтын системийн класс диаграм 47
2.3 Нэвтрэх үеийн дараалын диаграмм 48
2.4 Файл байршуулах үеийн дараалын диаграмм 49
2.5 Нөөц хуваалцах үеийн дараалын диаграмм 50

3.1 Системийн архитектур 52
3.2 Хувийн үүлэн хадгалалтын системийн ERD 54
3.3 Анхны тохиргооны дэлгэц 55
3.4 Нэвтрэх дэлгэц 55
3.5 Энгийн хэрэглэгчийн үндсэн хуудас 56
3.6 Админ хэрэглэгчийн нүүр хуудас 56

Хүснэгтийн жагсаалт

1.1 Google Drive, Nextcloud, ownCloud oCIS системүүдийн нэгдсэн үнэлгээ . 16

2.1 UC-001 юзкейсийн тодорхойлолт 24
2.2 UC-002 юзкейсийн тодорхойлолт 25
2.3 UC-003 юзкейсийн тодорхойлолт 26
2.4 UC-004 юзкейсийн тодорхойлолт 27
2.5 UC-005 юзкейсийн тодорхойлолт 28
2.6 UC-006 юзкейсийн тодорхойлолт 29
2.7 UC-007 юзкейсийн тодорхойлолт 30
2.8 UC-008 юзкейсийн тодорхойлолт 31
2.9 UC-009 юзкейсийн тодорхойлолт 32
2.10 UC-010 юзкейсийн тодорхойлолт 33
2.11 UC-011 юзкейсийн тодорхойлолт 34
2.12 UC-012 юзкейсийн тодорхойлолт 35
2.13 UC-013 юзкейсийн тодорхойлолт 36
2.14 UC-014 юзкейсийн тодорхойлолт 37
2.15 UC-015 юзкейсийн тодорхойлолт 38
2.16 UC-016 юзкейсийн тодорхойлолт 39
2.17 UC-017 юзкейсийн тодорхойлолт 40
2.18 UC-018 юзкейсийн тодорхойлолт 41
2.19 UC-019 юзкейсийн тодорхойлолт 42
2.20 UC-020 юзкейсийн тодорхойлолт 43
2.21 UC-021 юзкейсийн тодорхойлолт 44

Удиртгал

Өнөөгийн дижитал эрин үед үүлэн хадгалалт (Cloud Storage) нь хэрэглэгчдийн өдөр тутмын амьдралын салшгүй хэсэг болжээ. Хэрэглэгчид зураг, бичлэг, ажлын болон хувийн чухал файлуудаа интернетийн тусламжтайгаар хаанаас ч хандаж ашиглах шаардлагатай байдаг. Энэхүү хэрэгцээг хангахын тулд Google Drive, iCloud, OneDrive зэрэг төвлөрсөн үүлэн хадгалалтын үйлчилгээ өргөн хэрэглэгддэг болсон.

Гэвч эдгээр үйлчилгээний хувьд хэрэглэгчийн өгөгдөл тухайн үйлчилгээ үзүүлэгч компанийн сервер дээр хадгалагддаг бөгөөд хэрэглэгч өөрийн мэдээллийн бүрэн эзэмшил, хяналтыг алдах эрсдэлтэй байдаг. Мөн хадгалах багтаамж нэмэгдэх тусам тогтмол төлбөр төлөх шаардлага гардаг нь урт хугацаанд хэрэглэгчдэд эдийн засгийн дарамт учруулж болзошгүй.

Иймд хэрэглэгч өөрийн эзэмшлийн техник хангамжийг ашиглан өгөгдлөө хадгалах боломжтой, өөрөөр хэлбэл өөрийн сервер дээр суурилсан хувийн үүлэн хадгалалтын системийг хөгжүүлэх нь чухал асуудал болж байна. Энэхүү ажлын хүрээнд хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах, веб болон мобайл клиентээр дамжуулан удирдах боломжтой, хөнгөн бүтэцтэй (lightweight) хувийн үүлэн хадгалалтын системийг хөгжүүлэхийг зорьж байна.

Зорилго

Энэхүү төслийн зорилго нь хэрэглэгч өөрийн эзэмшлийн техник хангамжид суурилсан, өөрөө байршуулж (self-hosted) ажиллуулах боломжтой хувийн үүлэн хадгалалтын системийг зохион бүтээж хэрэгжүүлэхэд оршино. Тус систем нь хэрэглэгчдэд веб болон мобайл клиент ашиглан өөрийн сервер дээр хадгалагдсан файлуудыг удирдах, хандах боломжийг олгоно.

Төслийн зорилтууд

Дэвшүүлсэн зорилгод хүрэхийн тулд дараах зорилтуудыг хэрэгжүүлнэ. Үүнд:

1. Хувийн үүлэн хадгалалтын системүүдийн архитектур болон технологийн харьцуулсан судалгаа хийх;
2. Нөөц багатай төхөөрөмж (Raspberry Pi зэрэг) дээр ажиллах боломжтой хөнгөн серверийн программ хангамжийг зохион бүтээх;
3. Файл хадгалах, татах, зохион байгуулах үндсэн үйлдлүүдийг дэмжих серверийн REST API хөгжүүлэх;

4. Хэрэглэгч веб хөтөч ашиглан файлуудаа удирдах боломжтой веб клиент аппликейшн хөгжүүлэх;
5. iOS үйлдлийн системд зориулсан хэрэглэгчийн мобайл аппликейшн боловсруулах;
6. Хэрэглэгч өөрийн серверийг интернэт орчинд байршуулан алсаас хандах боломжийг бүрдүүлэх архитектурын шийдлийг боловсруулах.

Судлах үндэслэл

Сүүлийн жилүүдэд үүлэн хадгалалтын хэрэглээ огцом өсөж, 2025 оны байдлаар дэлхий даяар үүлэн хадгалалтын хэрэглэгчдийн тоо 2.3 тэрбумыг [1] давсан байна. Гэвч төвлөрсөн үүлэн үйлчилгээний хэрэглээ өсөхийн хэрээр дараах асуудлууд тулгарч байна.

1. **Өгөгдлийн эзэмшил ба нууцлал**: Хэрэглэгчийн мэдээлэл гуравдагч этгээдийн сервер дээр хадгалагддаг тул өгөгдлийн бүрэн хяналт хэрэглэгчид байхгүй.
2. **Тогтмол үйлчилгээний зардал**: Үүлэн хадгалалтын үйлчилгээний багтаамж нэмэгдэх тусам хэрэглэгч тогтмол төлбөр төлөх шаардлагатай болдог.

Иймд хэрэглэгч өөрийн техник хангамжийг ашиглан өгөгдлөө хадгалах боломжтой, өөрөөр хэлбэл self-hosted хувийн үүлэн системийг хөгжүүлэх нь технологийн болон практик ач холбогдолтой юм.

Судлагдсан байдал

Хувийн үүлэн хадгалалтын системийн чиглэлээр Nextcloud, Seafile, Syncthing зэрэг нээлттэй эхийн төслүүд өргөн хэрэглэгддэг. Эдгээр системүүд нь хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах боломжийг олгодог боловч ихэвчлэн том хэмжээний системийн архитектуртай, олон төрлийн нэмэлт функцүүдийг агуулсан байдаг.

Тухайлбал, Nextcloud нь хамтын ажиллагаа, календарь, мэйл зэрэг олон төрлийн үйлчилгээг нэг дор багтаасан цогц платформ бол Seafile нь файл синхрончлолын өндөр гүйцэтгэлд төвлөрсөн систем юм. Гэвч эдгээр системүүдийг бага нөөцтэй төхөөрөмж дээр ажиллуулахад тохиргоо хийхэд төвөгтэй, системийн шаардлага өндөр байх зэрэг сул талууд ажиглагддаг.

Иймд энэхүү төслөөр илүү хөнгөн бүтэцтэй, хувийн хэрэглээнд зориулсан үүлэн хадгалалтын системийг зохион бүтээхийг зорьсон.

Шинэлэг тал

Хувийн үүлэн хадгалалтын системийн шинэлэг тал:

1. **Хөнгөн архитектур**: Нөөц багатай төхөөрөмжүүд дээр ажиллах боломжтой энгийн бөгөөд үр ашигтай серверийн архитектур ашиглана.
2. **Self-hosted шийдэл**: Хэрэглэгч өөрийн сервер дээр системийг байршуулж өгөгдлөө бүрэн хянах боломжтой.
3. **Олон төрлийн клиент**: Веб болон мобайл клиент ашиглан файлуудад хандах боломжтой.

Технологийн ач холбогдол

Энэхүү систем нь технологийн хувьд дараах ач холбогдолтой:

1. **Өгөгдлийн бүрэн эзэмшил**: Хэрэглэгч өөрийн сервер дээр өгөгдлөө хадгалснаар мэдээллийн бүрэн хяналтыг хадгална.
2. **Хөнгөн серверийн шийдэл**: Орчин үеийн асинхрон серверийн архитектур ашиглан өндөр гүйцэтгэлтэй систем бий болгоно.
3. **Олон платформын клиент**: Веб болон мобайл клиентүүдээр дамжуулан хэрэглэгчийн хүртээмжийг нэмэгдүүлнэ.

Хамрах хүрээ

Тус төсөл нь дараах хүрээг хамарна:

- Өөрийн өгөгдлийн нууцлалыг эрхэмлэдэг хувь хүн болон гэр бүлийн хэрэглэгчид;
- Өндөр өртөг бүхий үүлэн үйлчилгээнээс татгалзаж, өөрийн нөөцийг ашиглах хүсэлтэй технологи сонирхогчид;

БҮЛЭГ 1

Хувийн үүлэн хадгалалтын системийн тухай онол, арга зүйн судалгаа

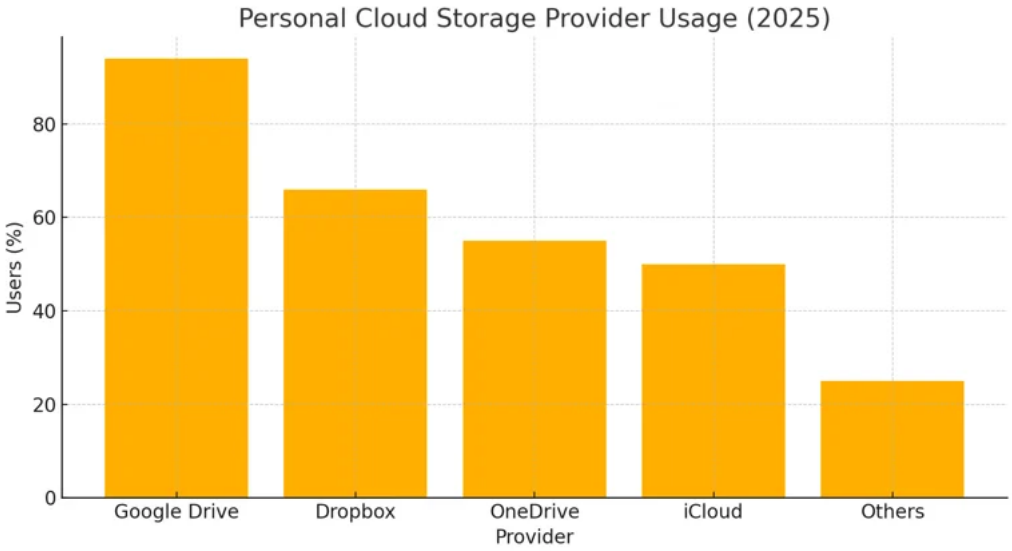
1.1 Үүлэн хадгалалтын системийн ерөнхий ойлголт

Үүлэн хадгалалт (*Cloud Storage*) нь өгөгдлийг локал төхөөрөмж дээр бус, сүлжээгээр холбогдсон алсын сервер дээр хадгалах технологи юм. Хэрэглэгч интернэтээр дамжуулан өөрийн файлуудад хаанаас ч, ямар ч төхөөрөмжөөс хандах боломжтой байдаг. Энэхүү технологи нь өгөгдлийг төвлөрсөн байдлаар хадгалах, нөөцлөх, хуваалцах зэрэг давуу талуудыг олгодог.

Үүлэн хадгалалтын системийг ерөнхийд нь дараах гурван төрөлд ангилдаг:

- **Нийтийн (Public Cloud)** – Гуравдагч байгууллагын эзэмшлийн сервер дээр өгөгдлийг хадгалдаг (жишээ нь: Google Drive, Dropbox). Хэрэглэхэд хялбар боловч өгөгдлийн хяналт харьцангуй бага байдаг.
- **Хувийн (Private Cloud)** – Хэрэглэгч өөрийн сервер дээр системийг ажиллуулж, өгөгдлөө бүрэн хянах боломжтой. Аюулгүй байдал болон нууцлал өндөр боловч тохиргоо, удирдлага шаарддаг.
- **Холимог (Hybrid Cloud)** – Нийтийн болон хувийн үүлэн системийг хослуулсан хэлбэр. Уян хатан байдал сайтай боловч архитектурын хувьд илүү төвөгтэй.

Сүүлийн жилүүдэд өгөгдлийн нууцлал, аюулгүй байдлын асуудал нэмэгдэж байгаатай холбоотойгоор хувийн үүлэн хадгалалтын системд анхаарах хандлага нэмэгдэж байна. Хэрэглэгч өөрийн өгөгдлийг бүрэн хянах, гуравдагч этгээдээс хамааралгүй байх шаардлага нь ийм төрлийн системийг хөгжүүлэх үндэслэл болж байна. 2025 оны судалгаагаар [1] дэлхий дээр нийт 2.3 тэрбум хүн үүлэн хадгалалтын үйлчилгээг гуравдагч системээс авч байгаа гэсэн судалгаа гарсан. Үүний хэрэглэгчдийн ашиглалтын үзүүлэлт болон заах зээлийн хуваалт нь Google Drive 94%, Dropbox 66%, OneDrive 55%, iCloud 50% (хэрэглэгчид эдгээр платформуудыг зэрэг хэрэглэсэн байна). Зураг 1.1-т хэрэглээний түвшинг графикаар харуулав.



Зураг 1.1: Хэрэглэгчдийн ашиглаж буй үүлэн хадгалалтын үйлчилгээний үзүүлэлт

Судалгааны үр дүнгээс үзэхэд хэрэглэгчдийн 55% дор хаяж 3 үүлэн хадгалалтын үйлчилгээг зэрэг хэрэглэж байна. Хүмүүс ихэвчлэн файлуудаа платформуудад хуваан, тухайн үйлчилгээний онцлогт таарууулан ашигладаг. Google Photos-ийг зураг хадгалахдаа ашиглаж, баримт бичгийг Dropbox, iPhone утасныхаа өгөгдлийг iCloud дээр хадгалах нь түгээмэл ажиглагдаж байна. Хэрэглэгчдийн дунд өгөгдлийн аюулгүй байдлын халдлага болон асуудал энэхүү салбарт хамгийн том асуудлуудын нэгт орж байгаа бөгөөд үүнд зориулж үйлчилгээ үзүүлэгч байгууллагууд илүү найдвартай шифрлэлт, хоёр шатлалт баталгаажуулалт гэх мэт хамгаалалтын шийдлүүдийг санал болгож байна. 2024 оны Англи улсад хэрэглэгчдээс авсан судалгаагаар үүлэн хадгалалтын хэрэглэгчдийн 43% нь тухайн үүлэн хадгалалтын үйлчилгээ үзүүлэгч байгууллага нь өөрсдийнх нь хувийн өгөгдлийг шинжилдэг гэдэгт санаа зовнидог гэсэн хариулт өгсөн байна. Үүлэн хадгалалтын үйлчилгээнүүд нь ойрын ирээдүйд хиймэл оюунтай ашиглан файл системийн автомат цэгцлэлт, ухаалаг хайлт, өгөгдөл шошгожуулах, олон платформ дээр дэмжигдэх байдал, сүлжээгүй үед хэрхэн sync хийгдэх гэсэн зүйлс дээр нэмэлт сайжруулалт хийгдэн өрсөлдөөн үүсэх төлөвтэй байгаа ажээ. **Энэхүү дипломын ажлаар миний бие хувийн гэсэн ангилалд багтах үүлэн хадгалалтын системийг хөгжүүлэхийг зорьж байгаа юм.**

1.2 Ижил төст системийн судалгаа

Энэ хэсгийн зорилго нь personal cloud storage системийн зохиомжид бодитой нөлөө үзүүлэх ижил төст платформуудыг харьцуулж, тэдгээрийн архитектур болон шийдлийн логикоос хэрэгжүүлэхүйц дүгнэлт гаргах явдал юм. Судалгаанд cloud storage ба file syncing гэсэн хоёр чиглэлийг ерөнхийд нь ялган авч үзсэн боловч дипломын ажлын үндсэн зорилт нь **personal cloud storage system** хөгжүүлэх тул гүн шинжилгээг cloud storage ангиллын төлөөлөгчдөд төвлөрүүлэв.

Тодруулбал Google Drive (public cloud), Nextcloud болон ownCloud Infinite Scale (self-hosted cloud) системүүдийг сонгон, дараах асуудлууд дээр төвлөрч судаллаа: архитектурын суурь давхарга, өгөгдлийн хяналт ба бодлогын удирдлага, синхрончлолын ажиллагааны шийдэл, аюулгүй байдал ба ажиллагааны төвөгшил. Харин Apple iCloud, Microsoft OneDrive, Seafile, Syncthing/Resilio Sync зэрэг платформуудыг зөвхөн судалгааны хүрээ, контекстийг тодорхойлох түвшинд дурдав.

1.2.1 Судалгааны үнэлгээний шалгуур

Сонгосон cloud storage системүүдийг дараах шалгууруар үнэлэв.

1. **Системийн зорилго**: тухайн платформ ямар үндсэн асуудлыг шийдэхэд төвлөрдөг вэ.
2. **Архитектурын бүтэц**: өгөгдөл хадгалах болон удирдах үндсэн давхаргууд хэрхэн зохион байгуулагдсан бэ.
3. **Өгөгдлийн хяналт**: өгөгдөл болон хандалтын бодлогыг хэрэглэгч хэр хэмжээнд өөрөө удирдах вэ.
4. **Аюулгүй байдал ба сэргээх боломж**: шифрлэлт, хувилбарын түүх, алдагдсан өгөгдөл сэргээх чадвар.
5. **Ашиглалт ба өргөтгөл**: өдөр тутмын хэрэглэхэд хялбар байдал болон системийг цааш өргөтгөх боломж.

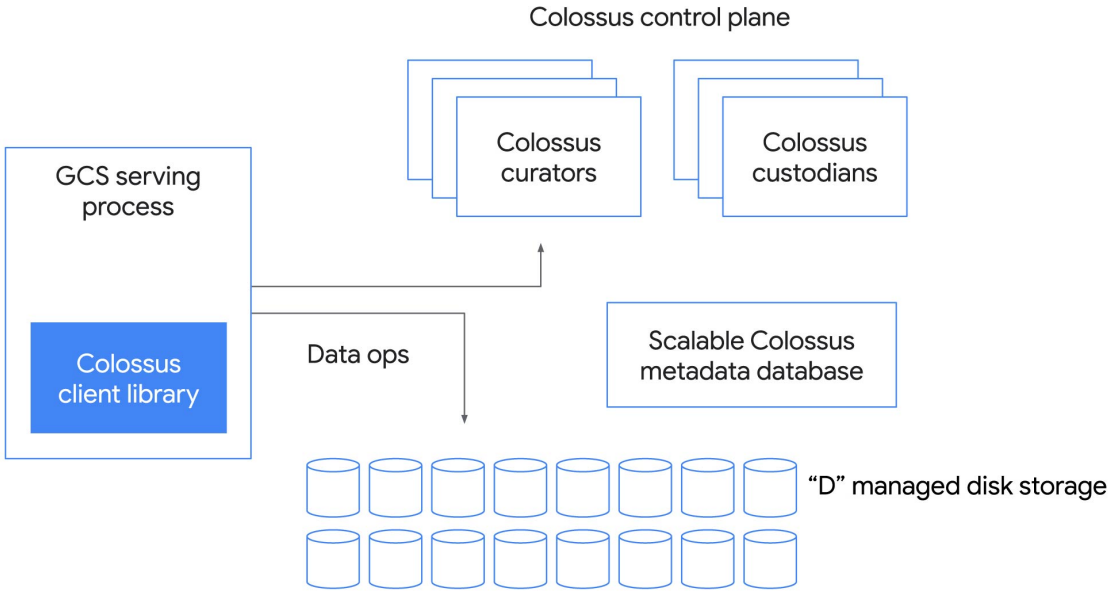
1.2.2 Google Drive: Нийтийн Cloud Storage-ийн Төлөөлөл

Google Drive нь personal cloud storage системтэй шууд ижил ангилалд хамаарахгүй ч, том хэмжээний cloud storage үйлчилгээг хэрхэн найдвартай ажиллуулж болдгийг харуулах сайн жишээ юм. Энэ платформын үнэ цэнэ нь зөвхөн файл хадгалахдаа биш, харин Google Workspace-ийн баримт бичиг, хамтын ажиллагаа, хуваалцах эрх, хайлт, хувилбарын түүх зэрэг олон урсгалыг нэг экосистемд уялдуулан тогтвортой

ажиллуулдгаараа илэрдэг. Тиймээс энэхүү судалгаанд Google Drive-ийг “боловсорсон public cloud storage систем ямар шинжтэй байх ёстой вэ” гэдэг өнцгөөс авч үзэв.

Google-ийн тайлбарласнаар Drive зэрэг үйлчилгээний ард Colossus, Spanner, Borg гэсэн нийтлэг суурь давхаргууд ажилладаг [2]. Энд Colossus нь файлын үндсэн хадгалалтыг, Spanner нь глобал хэмжээнд нийцтэй мета өгөгдлийн удирдлагыг, Borg нь тооцооллын нөөцийн зохион байгуулалтыг дэмждэг. Энэ нь хэрэглэгчийн хувьд нэг энгийн файл хадгалах үйлчилгээ мэт харагдавч, цаанаа олон дата төв, олон машин, тасралтгүй ачаалал, алдааны үед ч ажиллагаагаа үргэлжлүүлэхээр зохион байгуулагдсан систем гэсэн үг юм. Google Workspace-ийн аюулгүй байдлын баримт бичигт өгөгдлийг дамжуулах үед болон хадгалж байх үед шифрлэдэг, мөн өгөгдлийг олон диск, олон сервер, олон дата төв дээр ойролцоо хугацаанд давхар хадгалж өгөгдөл алдах эрсдэлийг бууруулдаг гэж тайлбарласан байдаг [3].

Өгөгдлийн хяналтын хувьд Google Drive нь self-hosted систем шиг бүрэн өмчлөл өгөхгүй ч байгууллагын бодлогын түвшний хэд хэдэн чухал тохиргоог санал болгодог. Жишээлбэл, Google Workspace админ нь тодорхой төрлийн өгөгдлийг АНУ, Европын Холбоо, эсвэл бүсийн онцгой шаардлагагүй гэсэн сонголтоор байршуулах боломжтой [4]. Мөн Client-side Encryption (CSE)-ийг ашигласнаар байгууллага өөрийн шифрлэлтийн түлхүүрийг удирдаж, өгөгдлийг Google-ийн үүлэнд хүрэхээс өмнө шифрлэх боломжтой. Ингэснээр Google серверүүд болон гуравдагч талууд тухайн түлхүүргүйгээр өгөгдлийг тайлж унших боломжгүй болдог [5]. Иймээс Google Drive-ийн хяналтын загварыг “платформын өмчлөл Google-д, харин бодлогын нарийвчилсан удирдлага тодорхой түвшинд хэрэглэгч байгууллагад” гэсэн хэлбэртэй гэж ойлгож болно.



Зураг 1.2: Colossus control plane-ийн ерөнхий бүтэц (эх сурвалж: Google Cloud Blog) [2]

Зураг 1.2-т Colossus-ийн control plane болон өгөгдлийн урсгалыг энгийн байдлаар харуулжээ. Зүүн талын client library нь үйлчилгээний хүсэлтийг Colossus руу дамжуулдаг. Дээд талд байрлах curator болон custodian төрлийн бүрэлдэхүүнүүд нь мета өгөгдөл, хадгалалтын зохион байгуулалт, тэнцвэржүүлэлт, нөхөн сэргээх зэрэг удирдлагын ажлыг хариуцна. Харин доод талын managed disk storage нь бодит өгөгдөл хадгалах хэсэг юм. Өөрөөр хэлбэл зураг нь удирдлагын үйлдэл ба өгөгдөл дамжуулах үйлдлийг салгавал системийг их хэмжээнд өргөтгөж, нэг хэсэгт асуудал гарсан ч бүхэл үйлчилгээ саатах эрсдэлийг багасгаж болдгийг харуулна [2]. Энэ нь personal cloud системийн хувьд ч хэрэгтэй санаа бөгөөд мета өгөгдөл, синк логик, файлын бодит хадгалалтыг нэг дор хэт нягт холбохгүй байх нь зөв шийдэл гэдгийг сануулна.

Google Drive-ийн хэрэглэгчийн түвшний хүчтэй тал нь синхрончлол болон сэргээх урсгал дээр илүү тод харагддаг. Drive API нь өөрчлөлтийн жагсаалтыг start page token-оос эхлэн дараалаар авах, өөрчлөлт гарсныг 'changes.watch'-аар мэдэгдэх, том хэмжээний файлыг тасалдсан тохиолдолд resumable upload-аар үргэлжлүүлэх, мөн файлын revision-уудыг удирдах боломж олгодог [6, 7, 8, 9, 10]. Иймээс Google Drive-ээс авах хамгийн чухал сургамж нь "файл хадгалах" боломж дангаараа хангалтгүй, харин өөрчлөлт мөрдөх, тасалдсан дамжуулалтыг сэргээх, хувилбарын түүх хадгалах зэрэг урсгалууд системийн үндсэн хэсэг байх ёстой гэсэн санаа юм.

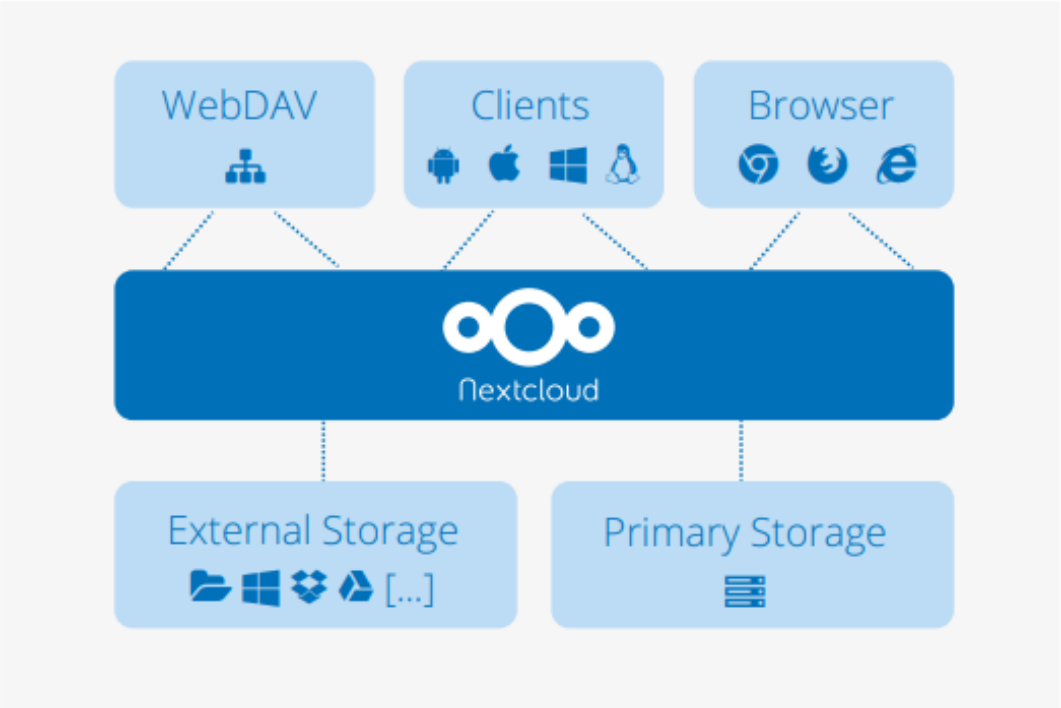
Дүгнэж хэлбэл Google Drive нь өгөгдлийн бүрэн өмчлөлөөс илүүтэйгээр production түвшний найдвартай ажиллагаа, синк урсгалын боловсронгуй байдал, аюулгүй байдлын бодлогын зохион байгуулалтаараа энэ судалгаанд үнэ цэнтэй жишиг болж байна. Харин энэхүү дипломын ажлын зорилго нь ийм төрлийн ажиллагааны давуу талыг авч, self-hosted орчинд хэрэглэгчийн өгөгдлийн хяналтыг илүү өндөр түвшинд хадгалах шийдэл боловсруулахад оршино.

1.2.3 Nextcloud: Self-Hosted Personal Cloud-ийн Төлөөлөл

Nextcloud нь self-hosted personal cloud-ийн хамгийн төлөөлөхүйц шийдлүүдийн нэг юм. Үүний гол ялгарах тал нь public cloud үйлчилгээ шиг файл хадгалах, хуваалцах, олон төхөөрөмжөөс хандах боломжийг өгдөг атлаа системийн эзэмшил, байршуулалт, бодлогын хяналтыг хэрэглэгч эсвэл байгууллагын талд үлдээдэгт оршино. Nextcloud Files-ийн албан ёсны танилцуулгад веб интерфэйс, desktop client, mobile app-аар хаанаас ч хандах боломжтой бөгөөд гуравдагч тал руу өгөгдөл алдахгүйгээр full control хадгална гэж тодорхойлсон байдаг [11]. Иймээс Nextcloud-ийг энэ судалгаанд "public cloud-ийн хэрэглээний мэдрэмжийг self-hosted орчинд ойртуулсан шийдэл" гэж үзэж болно.

Түүхэн хувьд Nextcloud нь ownCloud-той шууд холбоотой. Nextcloud-ийн албан ёсны 2016 оны зарлалд энэхүү төсөл нь ownCloud-ийн үндсэн үүсгэн байгуулагчид, тэр дундаа ownCloud төслийн үүсгэн байгуулагч Frank Karlitschek-ийн оролцоотойгоор ownCloud-оос салаалан үүссэн fork гэж тодорхой дурдсан байдаг [12]. Иймээс Nextcloud болон ownCloud-ийг бүрэн хамааралгүй хоёр систем гэж үзэхээсээ илүү, нэг ижил гаралтай боловч дараа нь өөр архитектур, өөр бүтээгдэхүүний чиглэлээр хөгжсөн хоёр салбар гэж ойлгох нь зөв юм.

Архитектурын түвшинд Nextcloud нь хэрэглэгчийн хандалтын давхарга болон хадгалалтын давхаргын дунд байрлах зохион байгуулалттай. Browser, desktop/mobile client, мөн WebDAV зэрэг хандалтын сувгууд нэг Nextcloud сервер рүү холбогдож, цаанаа primary storage болон external storage-уудтай ажилладаг [13, 14]. Энэ нь хэрэглэгчийн хувьд файлууд нэг л цэгт төвлөрсөн мэт харагдах боловч системийн админ талд локал диск, object storage, SMB/CIFS, SFTP, WebDAV зэрэг өөр өөр backend-ийг холбох уян хатан боломж олгодог [14].

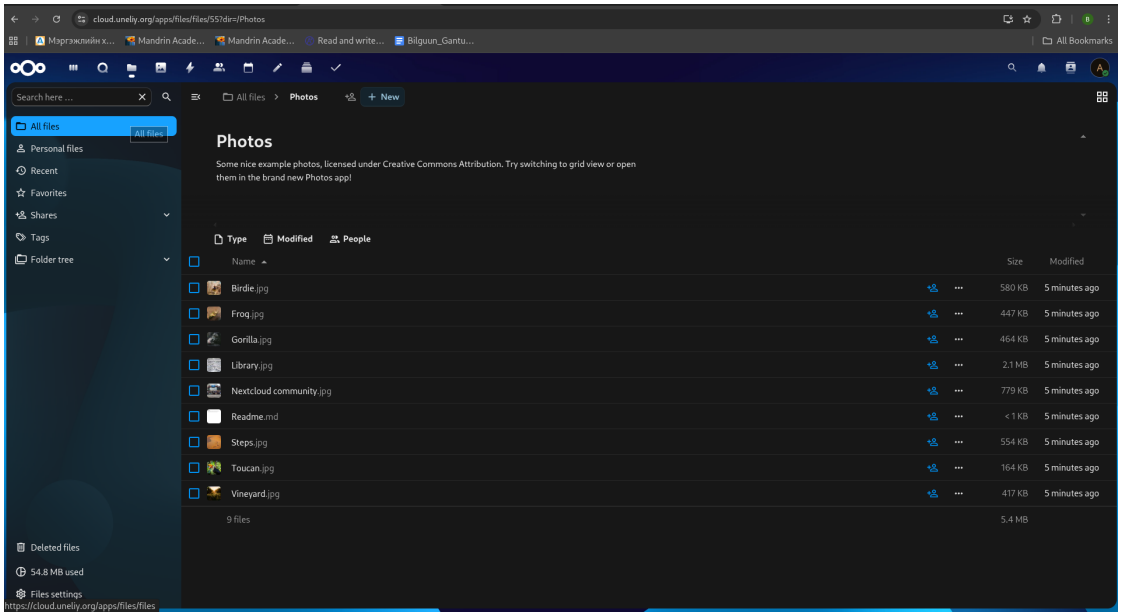


Зураг 1.3: Nextcloud-ийн хандалтын ба хадгалалтын давхаргын ерөнхий зураглал (эх сурвалж: Nextcloud Architecture Whitepaper) [13]

Зураг 1.3-т Nextcloud нь хэрэглэгчийн талын олон төрлийн интерфэйсийг доорх хадгалалтын эх үүсвэрүүдтэй холбодог "зууч давхарга" хэлбэрээр харагдана. Дээд хэсгийн browser, client, WebDAV нь хэрэглэгчид харагдах тал бол доод хэсгийн primary storage болон external storage нь бодит өгөгдөл байрлах орчин юм. Энэ зураг personal cloud системд дараах чухал санааг өгнө: хэрэглэгчийн интерфэйсийг хадгалалтын физик байршлаас салгаж чадвал нэг талаас хэрэглээний туршлага жигд, нөгөө талаас дэд бүтцийн сонголт уян хатан хэвээр үлдэнэ.

Өгөгдлийн хяналтын хувьд Nextcloud нь Google Drive-тай харьцуулахад илүү шууд бөгөөд нарийн удирдлага өгдөг. Админ нь external storage mount-уудыг хэрэглэгч, бүлэг, mount option-оор нь ялган тохируулж, хуваалцах эсэх, read-only эсэх, encryption хэрэглэх эсэхийг хүртэл тус бүрээр нь зааж болно [14]. Үүн дээр File Access Control механизмыг ашиглавал дүрэм биелсэн үед тухайн файлыг upload хийх, өөрчлөх, устгах, татах, бүр клиентээр sync хийхийг хүртэл хориглох боломжтой [15]. Аюулгүй байдлын хувьд Nextcloud нь Server-Side Encryption болон End-to-End Encryption-ийг ялгаж авч үздэг; Server-Side Encryption нь файлыг сервер дээр болон external storage дээр хамгаалдаг бол End-to-End Encryption нь файлыг client талд шифрлэдэг учраас серверийн админ хүртэл агуулгыг тайлж үзэх боломжгүй байдаг [16]. Энэ нь personal

cloud системд хэрэгтэй маш чухал ялгаа бөгөөд “өгөгдөл миний сервер дээр байна” гэдэг нь дангаараа хангалтгүй, ямар түвшний хэн өгөгдлийг уншиж чадах вэ гэдгийг архитектурын түвшинд шийдэх шаардлагатайг харуулна.



Зураг 1.4: Docker орчинд ажиллуулсан Nextcloud-ийн веб интерфэйсийн дэлгэцийн зураг

Зураг 1.4 нь миний бие Nextcloud-ийг Docker орчинд амжилттай ажиллуулж турш-сан веб интерфэйсийн жишээ юм. Зурагт харагдаж буйгаар файлуудыг жагсаах, зураг хадгалах, хуваалцах, үндсэн navigation ашиглах зэрэг өдөр тутмын үүлэн хадгалалтын гол үйлдлүүд хэрэглэгчийн хувьд ойлгомжтой, бүрэн хэрэглэхүйц түвшинд ажиллана. Энэ нь Nextcloud self-hosted шийдэл боловч хэрэглэгчийн туршлагын хувьд public cloud үйлчилгээтэй төстэй мэдрэмж өгч чаддгийг батална.

```
CONTAINER ID   NAME                          CPU %     MEM USAGE / LIMIT     MEM %
d92b16684685   nextcloud-aio-apache          0.03%     79.58MiB / 46.88GiB   0.17%
642a09d6396b   nextcloud-aio-nextcloud       0.03%     281.7MiB / 46.88GiB   0.59%
b02796f21b65   nextcloud-aio-imaginary       0.01%     225.6MiB / 46.88GiB   0.47%
ab9e15449795   nextcloud-aio-redis           0.27%     9.906MiB / 46.88GiB   0.02%
d53d07c76423   nextcloud-aio-database        3.91%     68.51MiB / 46.88GiB   0.14%
17fee3931d78   nextcloud-aio-whiteboard      0.01%     132.7MiB / 46.88GiB   0.28%
7cc917e82e7a   nextcloud-aio-notify-push     0.04%     5.34MiB / 46.88GiB    0.01%
134450ec6ec2   nextcloud-aio-talk            0.83%     126.9MiB / 46.88GiB   0.26%
07a7c84ccab0   nextcloud-aio-mastercontainer 0.42%     76.36MiB / 46.88GiB   0.16%
```

Зураг 1.5: Docker дээр ажиллуулсан Nextcloud AIO контейнерүүдийн санах ойн хэрэглээний дэлгэцийн зураг

Зураг 1.5-т миний Docker орчин дахь Nextcloud AIO deployment-ийн контейне-рүүдийн санах ойн хэрэглээг харуулсан. Тухайн зурагт байгаа `apache`, `nextcloud`,

`imaginary`, `redis`, `database`, `whiteboard`, `notify_push`, `talk`, `mastercontainer` зэрэг нийт контейнерүүдийн хэрэглээг нэгтгэн тооцоход ойролцоогоор 1006.6 MiB буюу бараг 1 GiB RAM ашиглаж байна. Энэ нь ердийн сервер орчинд боломжийн үзүүлэлт боловч Raspberry Pi зэрэг жижиг төхөөрөмж, эсвэл зөвхөн нэг хэрэглэгчийн хувийн хэрэглээнд зориулсан маш хөнгөн системийн хувьд өндөр шаардлага гэж үзэж болно. Өөрөөр хэлбэл Nextcloud нь боломж, өргөтгөл, collaboration талдаа хүчтэй боловч зөвхөн personal use-д чиглэсэн, бага нөөцтэй төхөөрөмж дээр ажиллах систем зохиох үед илүү минимал архитектур хэрэгтэй гэсэн дүгнэлтэд хүргэж байна.

Сэргээх болон ажиллагааны найдвартай байдлын талаас Nextcloud нь нэлээд боловсорсон механизмтай. Versions app нь файлын өмнөх хувилбаруудыг автоматаар хадгалж, хугацаа болон сул зайд тулгуурлан хуучин version-уудыг цэвэрлэдэг [17]. Trash bin нь устсан файлуудыг retention policy-аар тодорхой хугацаанд хадгалж, шаардлагатай үед буцаан сэргээх боломж өгдөг [18]. Харин файл эвдрэх, давхар өөрчлөлтөөс үүсэх асуудлыг бууруулахын тулд transactional file locking ашигладаг бөгөөд өндөр ачаалалтай орчинд Redis-ийг locking болон caching-д ашиглах нь ажиллагааг мэдэгдэхүйц тогтвортой болгодог [19, 20]. Гэхдээ эдгээр боломжийг найдвартай ажиллуулахын тулд database, cache, background job, TLS, backup policy, update lifecycle зэрэг ажиллагааны асуудлыг заавал урьдчилан төлөвлөх хэрэгтэй [21].

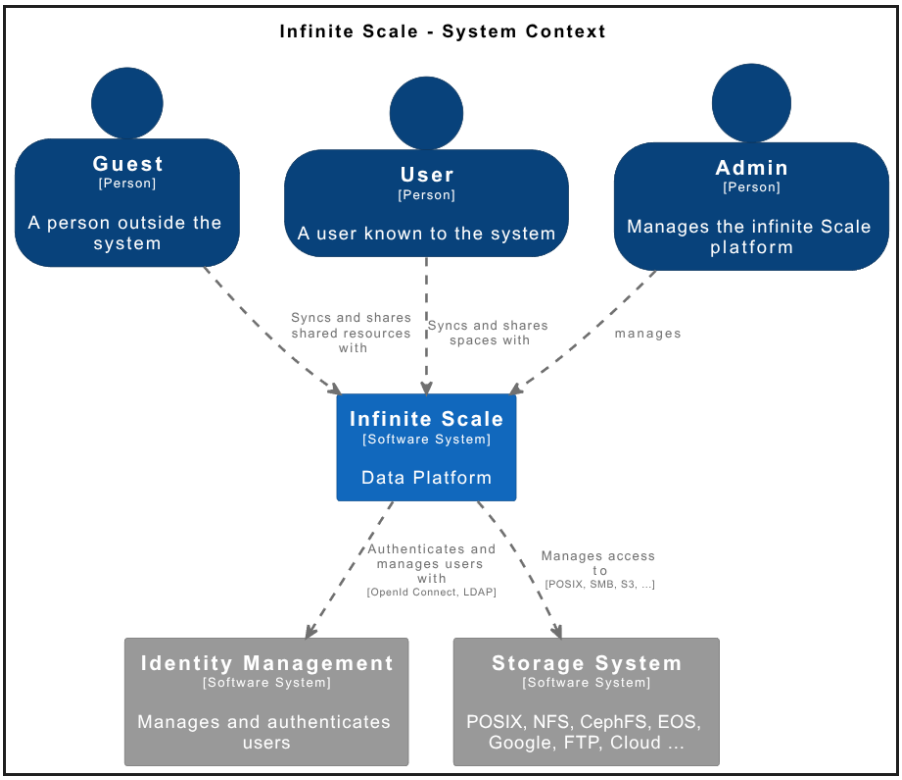
Дүгнэж хэлбэл Nextcloud нь personal cloud системийн үндсэн санаа болох "өгөгдөл өөрийн хяналтад байх ёстой" гэсэн шаардлагыг маш сайн хэрэгжүүлдэг жишээ юм. Гэвч миний Docker туршилтаас харахад ийм шийдэл нь олон боломж, олон үйлчилгээ дагалдуулж ирдэг тул нөөцийн хэрэглээ өсөх хандлагатай байна. Тиймээс энэхүү дипломын ажлын чиглэл нь Nextcloud-ийн өгөгдлийн хяналт, self-hosted зарчмыг хадгалах атлаа зөвхөн хувь хэрэглээнд зориулсан, жижиг төхөөрөмж дээр ч ажиллах илүү хөнгөн жинтэй cloud storage сервис боловсруулахад оршиж байна.

1.2.4 ownCloud Infinite Scale (oCIS): Микросервис Чиглэлийн Personal Cloud

ownCloud Infinite Scale (oCIS) нь self-hosted cloud storage-ийг уламжлалт ownCloud 10 загвараас өөр, илүү орчин үеийн аргаар шийдсэн платформ юм. ownCloud-ийн албан ёсны танилцуулгад Infinite Scale-ийг ownCloud-ийн cloud collaboration platform-ийн "next generation" гэж тодорхойлж, Go, Vue.js, three-tier architecture, cloud-native microservices зэрэг шинэ стек дээр тулгуурлан бүтээсэн гэж тайлбарласан байдаг

[22]. Энэ утгаараа oCIS-ийг зүгээр нэг өөр бүтээгдэхүүн гэж бус, харин ownCloud-ийн дараагийн үеийн архитектурын шинэчлэл гэж ойлгож болно. ownCloud-ийн архитектурын баримт бичигт ч Infinite Scale-ийг data platform, unified data access, cloud data ecosystems гэсэн ойлголтуудыг төвд нь тавьж зохиосон гэж тайлбарласан байдаг [23]. Өөрөөр хэлбэл энэ нь зөвхөн "файл хадгалах сервер" биш, харин өгөгдөл, мета өгөгдөл, хуваалцах бодлого, identity integration-ийг салангид боловч уялдаа холбоотой үйлчилгээ болгон зохион байгуулахыг зорьсон систем юм.

oCIS-ийн гол онцлог нь микросервис чиглэлийн логиктой атлаа хэрэглээнд хэт хүндрэлтэй байхгүйгээр эхлүүлэхийг зорьдогт оршино. ownCloud-ийн тайлбарласнаар default тохиргоонд бүх built-in service-үүдийг нэг процесс дотор 'ocis server' командаар эхлүүлж болдог, харин шаардлага өсөх үед эдгээрийг өөр node-ууд руу салган байршуулж scale-out хийх боломжтой [23]. Энэ санаа нь онолын хувьд маш сонирхолтой: жижиг deployment-оор эхэлж, дараа нь proxy, graph, WebDAV, sharing, storage зэрэг хэсгүүдийг тус тусад нь өргөтгөж болно. Nextcloud-тай харьцуулахад энд "applications over one storage server" гэхээсээ илүү "cooperating services over a shared identity and storage model" гэсэн архитектурын сэтгэлгээ давамгайлж байна.



Зураг 1.6: ownCloud Infinite Scale-ийн system context зураг (эх сурвалж: ownCloud docs) [23]

Зураг 1.6 нь ownCloud-ийн албан ёсны system context дүрслэл бөгөөд oCIS-ийн ерөнхий байрлалыг маш ойлгомжтой харуулна. Дээд хэсэгт guest, user, admin гэсэн үндсэн оролцогчид байна. Тэдгээр нь төвд байрлах Infinite Scale data platform-той харилцаж, уг платформ нь доод талдаа identity management болон storage system гэсэн хоёр чухал гадаад тулгууртай холбогдоно. Энэ зураг нь oCIS-ийг "дан файл хадгалах сервер" гэхээсээ илүү, хэрэглэгч, эрхийн удирдлага, хадгалалт гэсэн гурван талыг холбодог платформ гэж ойлгоход тохиромжтой.

Функциональ концепцийн хувьд oCIS-ийн хамгийн сонирхолтой санаа нь "Spaces" юм. ownCloud-ийн баримтад spaces-ийг хэрэглэгчийн харах файлын логик орон зай гэж тайлбарладаг бөгөөд access нь roles-оор удирдагдана, мөн space бүр quota, manager, syncability зэрэг шинжтэй байж болно [23]. Энэ нь personal storage, project drive, group space, incoming share зэрэг өөр өөр төрлийн өгөгдлийг нэг ижил загвараар авч үзэх боломж олгодог. Ийм abstraction нь personal cloud-ийг цааш өргөтгөхөд ашигтай ч, анхны шатны хэрэгжүүлэлт дээр заавал бүхэлд нь дуурайх шаардлагагүй. Гэхдээ "файлын мод"-оос илүү "логик орон зай + эрхийн загвар" гэсэн сэтгэлгээг авч үзэх нь маш хэрэгтэй сургамж юм.

Аюулгүй байдлын хувьд oCIS нь үйлчилгээ бүр дээр тус тусад нь нэвтрэх эрх шалгахаас илүү gateway төвтэй зохион байгуулалтыг ашигладаг. ownCloud-ийн security баримтад бүх хүсэлт proxy service-ээр дамждаг тул бусад service-үүдийг firewall-ийн ард нууж, attack surface-ийг бууруулдаг гэж тайлбарласан [24]. Мөн OpenID Connect дээр суурилсан identity integration нь жижиг instance дээр дотоод IDM ашиглах, илүү бодит орчинд гадаад identity provider-тай холбох хоёр өөр зам нээж өгдөг [24]. Extensions/service-үүд нь standalone service байдлаар API-аар харилцдаг тул буруу ажилласан нэг extension бүх storage-д шууд нөлөөлөх эрсдэл бага байдаг [24]. Энэ нь policy isolation болон service isolation-ийг зэрэг авч үздэгээрээ Nextcloud-аас ялгаатай.

ownCloud өөрсдөө availability and scalability баримтдаа ийм төрлийн modern microservices-based software-ийг single эсвэл distributed instance байдлаар ажиллуулахдаа шийдвэрүүдийг маш нухацтай гаргах хэрэгтэй гэж онцолсон байдаг [25]. Архитектурын хувьд scale-out, container-based deployment, service separation нь маш хүчтэй боловч бодит ажиллагаан дээр service discovery, monitoring, logging, reverse proxy, identity integration, shared storage зэрэг олон шинэ асуудлыг хамтад нь авчирна [23, 25]. Тиймээс oCIS нь хувийн үүлэн хадгалалтын системийг ирээдүйд enterprise эсвэл multi-tenant чиглэлд өргөтгөхөд оновчтой санаа өгдөг ч, зөвхөн нэг хэрэглэгч

эсвэл жижиг төхөөрөмж дээр ажиллах хөнгөн системийн хувьд хэт их архитектурын давхарга болох эрсдэлтэй.

Дүгнэж хэлбэл ownCloud Infinite Scale нь personal cloud-ийн “дараагийн үе”-ийн загварыг харуулдаг: gateway төвтэй хамгаалалт, spaces дээр суурилсан access model, identity federation, scale-out чадвар. Харин энэхүү дипломын ажлын өнцгөөс харахад эдгээр нь шууд хуулж хэрэгжүүлэх загвар биш, харин ирээдүйн өргөтгөлийн архитектурт ашиглаж болох гарын авлага юм. Өөрөөр хэлбэл oCIS-оос авах гол сургамж нь минимал хувийн системийг одоо хөнгөн байлгахын зэрэгцээ, цааш томрох үед service separation болон policy layer-уудыг хэрхэн нэмэхийг урьдчилан бодож загварчлах хэрэгтэй гэсэн санаа юм.

1.2.5 Нэгдсэн Харьцуулалт

Шалгуур	**Google Drive**	**Nextcloud**	**ownCloud oCIS**
Системийн зорилго	Хамтын ажиллагаа, синк, баримт бичгийн урсгалд хамгийн боловсорсон шийдэл.	Хувийн өгөгдлөө өөрийн сервер дээр төвлөрүүлж удирдахад хамгийн ойр шийдэл.	Томорч өргөжих self-hosted платформ руу чиглэсэн дараагийн үеийн шийдэл.
Архитектурын бүтэц	Маш хүчтэй distributed architecture-тай ч дотоод бүтэц нь хэрэглэгчид хаалттай.	Нэг сервер дээр ойлгомжтой ажиллуулж болдог ч нэмэлт сервисүүдтэй болох тусам хүндэрнэ.	Service-үүдээр сайн задарсан, өргөтгөхөд тохиромжтой ч анхнаасаа илүү нийлмэл.
Өгөгдлийн хяналт	Бодлогын түвшний хяналт сайн ч өгөгдлийн дэд бүтэц Google-ийн мэдэлд байна.	Өгөгдөл, storage, sharing, access rule-аа өөрөө удирдах боломж хамгийн өндөр.	Access model нь уян хатан боловч identity болон policy давхарга нь илүү төвөгтэй.
Аюулгүй байдал ба сэргээх боломж	Хувилбар, audit, recoverability маш сайн; enterprise түвшний найдвартай байдалтай.	Encryption, versioning, trash recovery сайн ч admin-ы зөв тохиргооноос их хамаарна.	Gateway болон policy төвтэй хамгаалалт хүчтэй ч зөв ажиллуулахад илүү их тохиргоо шаардаг.
Ашиглалт ба өргөтгөл	Хэрэглэхэд хамгийн хялбар, гэхдээ платформын хязгаарлалтаас бүрэн гарах боломжгүй.	Personal cloud-д шууд хэрэглэхэд хамгийн бодитой, харин нөөцийн хэрэглээ харьцангуй өндөр.	Ирээдүйд scale-out хийхэд хамгийн бэлэн, харин жижиг төхөөрөмж дээр хэт нарийн архитектур болж магадгүй.

Хүснэгт 1.1: Google Drive, Nextcloud, ownCloud oCIS системүүдийн нэгдсэн үнэлгээ

Хүснэгт 1.1-д Google Drive, Nextcloud, ownCloud oCIS гэсэн гурван төлөөлөх системийг нэг хүснэгтэд нэгтгэн харьцуулсан. Үүнээс аль систем нь public cloud шинж давамгайлж байна, аль нь self-hosted хэрэгцээнд ойр байна, мөн аль нь ирээдүйн өргөтгөлийн архитектурын санаа өгч байгааг нэг дор харах боломжтой.

Дүгнэж хэлбэл, энэхүү дипломын ажлын хувьд **Nextcloud** нь хувийн өгөгдлөө өөрийн хяналт дор ажиллуулах хамгийн ойр жишээ болно. **oCIS** нь системийг цааш томруулж, илүү олон service-тэй болгох үед ямар архитектур хэрэгтэйг харуулна. Харин **Google Drive**-аас синк, өөрчлөлт хянах, файл сэргээх урсгалыг хэрхэн найдвартай зохион байгуулах санааг авах нь хамгийн тохиромжтой юм.

1.3 Ашиглах технологиуд

Энэхүү дипломын ажлын систем нь хэрэглэгчийн өөрийн сервер дээр ажиллах, хөнгөн бүтэцтэй байх, бага нөөцтэй орчинд ч ашиглагдах, мөн веб болон мобайл клиентээр хандах боломжтой байх шаардлагатай. Иймээс технологийн сонголтыг зөвхөн түгээмэл байдлаар нь биш, харин **нөөцийн хэрэглээ**, **deployment-ийн хялбар байдал**, **хөгжүүлэлтийн найдвартай байдал**, **хэрэглэгчид хүрэх интерфейс** гэсэн хэрэгцээтэй уялдуулан хийв.

Rust хэлийг сервер талын хэрэгжүүлэлтэд сонгосон. Энэ системийн хамгийн чухал шаардлагын нэг нь жижиг сервер, хувийн компьютер, эсвэл нөөц багатай төхөөрөмж дээр ажиллах боломжтой байх явдал юм. Rust нь хөрвүүлэгддэг хэл учраас ажиллах орчны нэмэлт зардал багатай, нэг executable файл хэлбэрээр тараах боломжтой тул self-hosted системд тохиромжтой [26]. Мөн ownership, borrowing, lifetime зэрэг механизмаар санах ойтой холбоотой олон алдааг хөрвүүлэх үед илрүүлэх боломж олгодог [26]. Энэ нь файл хадгалах, хэрэглэгчийн эрх шалгах, файл байршуулах болон татах урсгал боловсруулах серверийн найдвартай ажиллагаанд чухал нөлөөтэй. Async/await ашиглан олон хүсэлтийг зэрэг боловсруулах боломж нь нэгэн зэрэг хэд хэдэн файлтай ажиллах үед тогтвортой суурь бүрдүүлнэ.

PostgreSQL өгөгдлийн санг системийн тогтвортой өгөгдлийг хадгалах үндсэн сан болгон ашиглана. Хэрэглэгчийн бүртгэл, файл болон хавтасны бүтэц, хадгалалтын хязгаар, хуваалцах эрх, системийн төлөв зэрэг өгөгдөл хоорондоо холбоотой байдаг тул хүснэгтэн бүтэцтэй, найдвартай өгөгдлийн сан шаардлагатай. PostgreSQL нь ийм төрлийн өгөгдлийг цэгцтэй зохион байгуулах, шаардлагатай үед нарийн шүүлт хийх, transaction ашиглан нэг үйлдлийн бүрэн бүтэн байдлыг хадгалахад тохиромжтой [27]. Иймээс серверийн хөнгөн архитектурыг эвдэхгүйгээр бат бөх өгөгдлийн суурь бүрдүүлнэ.

Хэрэглэгчийн талд системийг хоёр сувгаар ашиглах шаардлагатай гэж үзсэн. Нэгдүгээрт, ямар ч төхөөрөмжөөс веб хөтчөөр хандах боломжтой **веб аппликейшн** хэрэгтэй. Хоёрдугаарт, iPhone болон iPad дээр илүү шууд, төхөөрөмжид нийцсэн хэрэглээ өгөх **iOS native аппликейшн** шаардлагатай. Иймээс клиент талыг веб болон iOS гэсэн хоёр хэлбэрээр хөгжүүлнэ.

iOS аппликейшн-ийг **Swift** болон **SwiftUI** ашиглан хөгжүүлнэ. Энэ сонголтын гол шалтгаан нь Apple төхөөрөмж дээр native интерфэйс, тогтвортой ажиллагаа, системийн боломжуудтай илүү сайн уялдах явдал юм. Файл сонгох, урьдчилан харах, нэвтрэлт хадгалах, мобайл орчинд энгийн бөгөөд цэвэр интерфэйс өгөх зэрэгт native хөгжүүлэлт илүү тохиромжтой. Энэхүү дипломын ажлын зорилго нь олон платформд нэг дор туршихаас илүү, хэрэглэгчид шууд ашиглаж болох бодит iOS клиент боловсруулах тул Swift/SwiftUI нь зөв сонголт болно.

Веб аппликейшн-ийг **Angular** фреймворк ашиглан хөгжүүлнэ. Учир нь энэ системд нэвтрэх хэсэг, файлын жагсаалт, байршуулах болон татах урсгал, хэрэглэгчийн удирдлага, хуваалцах үйлдэл зэрэг хоорондоо холбоотой олон дэлгэц, төлөв, формын урсгал бий. Angular нь ийм төрлийн бүтэцтэй веб системийг цэгцтэй зохион байгуулахад тохиромжтой. TypeScript суурьтай тул кодын бүтэц тодорхой, томроход арчлахад хялбар байна. Веб клиент нь хэрэглэгчдэд ямар ч нэмэлт суулгалтгүйгээр системд хандах үндсэн гарц болох бол iOS клиент нь өдөр тутмын мобайл хэрэглээг дэмжинэ.

Ингэснээр технологийн сонголт бүхэлдээ системийн зорилготой уялдаж байна. **Rust** болон **PostgreSQL** нь хөнгөн боловч найдвартай серверийн үндэс суурийг бүрдүүлнэ. **Angular** нь удирдлагын болон файлын веб интерфэйсийг цэгцтэй хэрэгжүүлэхэд, **Swift/SwiftUI** нь iOS төхөөрөмж дээр энгийн, шууд хэрэглээ өгөхөд тохирно. Өөрөөр хэлбэл эдгээр технологиуд нь зөвхөн техник талаасаа боломжтой сонголт биш, харин энэхүү дипломын ажлын "хувийн, хөнгөн, өөрийн хяналттай үүлэн хадгалалтын систем" гэсэн хэрэгцээнд нийцсэн сонголтууд юм.

Бүлгийн дүгнэлт

Энэ бүлэгт үүлэн хадгалалтын системийн үндсэн ойлголт, хөгжлийн чиг хандлага, мөн ижил төст платформуудын архитектурын санааг судаллаа. Google Drive, Nextcloud, ownCloud oCIS системүүдийг харьцуулж үзэхэд public cloud шийдлүүд нь найдвартай ажиллагаа, синк, сэргээх урсгалаараа давуу боловч өгөгдлийн бүрэн хяналт хэрэглэгчийн талд байдаггүй нь харагдлаа. Харин self-hosted чиглэлийн шийдлүүд нь өгөгдлийн өмчлөл, хандалтын хяналтыг илүү сайн хангаж байгаа ч суулгалт, тохиргоо, нөөцийн хэрэглээний хувьд илүү анхаарал шаардаг байна.

Иймээс энэхүү дипломын ажлын хувьд хөнгөн бүтэцтэй, өөрийн сервер дээр ажиллах, хэрэглэгч өгөгдлөө өөрөө хянах боломжтой personal cloud систем боловсруулах чиглэл нь хамгийн оновчтой гэж дүгнэв. Мөн серверт Rust, веб клиентэд Angular,

iOS клиентэд Swift/SwiftUI ашиглах нь системийн хэрэгцээтэй нийцэж байгааг тодорхойлов. Дараагийн бүлэгт энэхүү дүгнэлтэд тулгуурлан системийн шаардлага, хэрэглэгчийн үүрэг, үндсэн үйл ажиллагааг нарийвчлан тодорхойлно.

БҮЛЭГ 2

Системийн шаардлага ба шинжилгээ

2.1 Системийн үйл ажиллагааны дэлгэрэнгүй

Энэхүү системийг хоёр тусдаа ажиллах хэсэгтэй байхаар төлөвлөнө. Нэг нь хэрэглэгчийн өдөр тутмын үйлдлүүдийг хариуцах үндсэн хэсэг байх бөгөөд нөгөө нь зөвхөн анхны тохиргоонд зориулагдсан тусгай хэсэг байна. Систем анхны тохиргоо хийгдээгүй үед зөвхөн анхны тохиргооны хэсэг ажиллах бөгөөд энэ зохион байгуулалт нь эхний тохиргооны эмзэг үйлдлүүдийг ердийн хэрэглээний урсгалаас салгана.

Анхны тохиргооны үеийн шаардлагад систем анхны тохиргоо хийгдсэн эсэхийг шалгах боломж орно. Үүний дараа эхний admin хэрэглэгчийн мэдээллийг оруулах бөгөөд үүнд хэрэглэгчийн нэр, и-мэйл, бүтэн нэр, нууц үг, нууц үгийн баталгаажуулалт, мөн өгөгдөл хадгалах үндсэн зам багтана. Нэмэлтээр системийн нийт хадгалалтын дээд хэмжээг тохируулах боломжтой байна. Тохиргоо амжилттай дууссаны дараа шаардлагатай хадгалалтын бүтэц үүсэж, анхны тохиргооны хэсэг цаашид идэвхгүй болохоор зохион байгуулна.

Анхны тохиргооны дараа хэрэглэгч веб аппликейшн эсвэл iOS аппликейшнээр дамжин нэвтрэх нэр, нууц үг ашиглан системд нэвтрэх боломжтой байна. Амжилттай нэвтэрсэн тохиолдолд систем хэрэглэгчийг баталгаажуулж, цаашдын үйлдлүүдийг гүйцэтгэх эрх олгоно. Үүний үндсэн дээр хэрэглэгч өөрийн профайл мэдээллийг харах, профайлын зураг шинэчлэх зэрэг үндсэн үйлдлүүдийг хийх боломжтой байна.

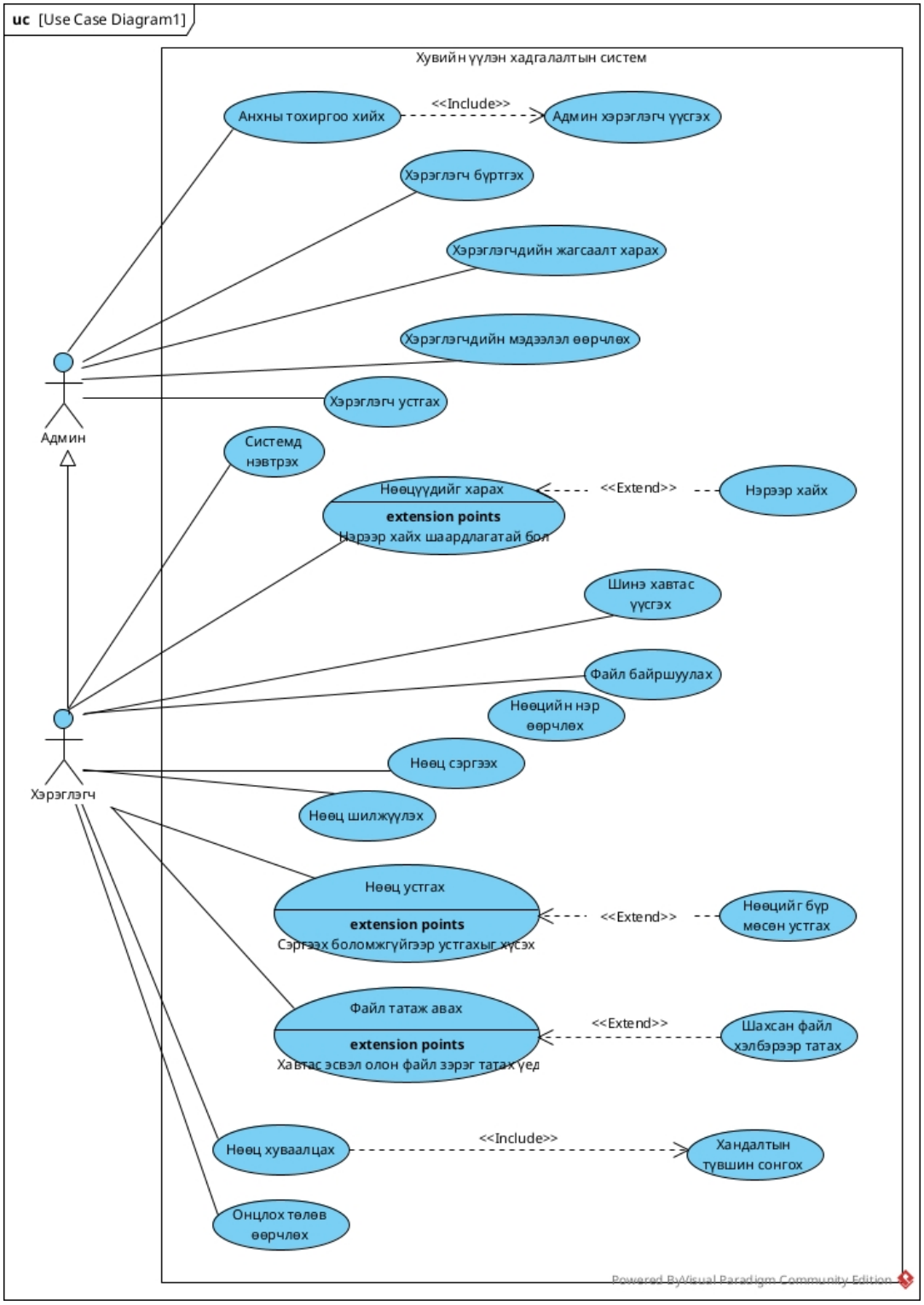
Admin хэрэглэгч нь системийн удирдлагын хэсгээр дамжин хэрэглэгчдийн бүртгэлийг удирдах шаардлагатай. Тэрээр бүх хэрэглэгчийг жагсааж харах, шинэ энгийн хэрэглэгч үүсгэх, хэрэглэгчийн үндсэн мэдээлэл болон хадгалалтын хязгаарыг шинэчлэх, мөн admin биш хэрэглэгчийг устгах боломжтой байна. Хэрэглэгч устах үед тухайн хэрэглэгчийн нэвтрэлтийн төлөв, профайлын зураг, хуваалцсан эрхүүд болон хувийн хадгалалтын бүтэц хамтад нь цэвэрлэгдэхээр шаардлага тодорхойлно.

Файлын сангийн логикийг “өөрийн эзэмшлийн орчин” болон “өөртэй нь хуваалцсан орчин” гэсэн хоёр үндсэн хэсэгтэй байна. Хэрэглэгч өөрийн хавтас, файлуудыг шаталсан бүтэцээр харах, дэлгэрэнгүй мэдээлэл авах, хандах эрхтэй өгөгдлүүд дотроос хайлт хийх, мөн устгасан файлууд, онцолсон файлууд, хуваалцсан өгөгдөл гэсэн тусгай харагдацуудаар ажиллах боломжтой байх ёстой. Ингэснээр өдөр тутмын хэрэглээний үндсэн урсгалууд нэгтгэсэн зохион байгуулалттай байна.

Үйлдлийн эрхийн хувьд эзэмшигч болон засварлагч эрхийг ялгаж үзэхээр төлөвлөнө. Засварлагч эрхтэй хэрэглэгч хуваалцсан хавтас дотор хавтас үүсгэх, файл байршуулах, зарим нэр өөрчлөх үйлдэл хийх боломжтой байх боловч эрхийн удирдлага, шилжүүлэх, устгах, сэргээх, онцлох зэрэг эзэмшигч төвтэй үйлдлүүдийг хийхгүй. Харин эзэмшигч хэрэглэгч өөрийн файл, хавтсан дээр бусдад харах эсвэл засварлах эрх олгох, устгасан өгөгдлөө сэргээх, бүр мөсөн устгах, мөн олон файл, хавтсыг нэг дор шахсан файл хэлбэрээр татах зэрэг өргөн удирдлагын боломжтой байна.

2.2 Usecase диаграмм

Зураг 2.1 дээр үзүүлж буй юзкейс диаграммд системийн үндсэн 2 хэрэглэгч болох админ болон энгийн хэрэглэгч гэсэн 2 оролцогчийг дүрслэв. Админ хэрэглэгч нь энгийн хэрэглэгчээс удамшиж байгаа ба энгийн хэрэглэгчийн бүх usecase-ийг хийх чадамжтай юм. Шаардлагатай юзкэйсүүдэд include болон extend харилцаануудыг нэмж оруулсан. Жишээлбэл анхны тохиргоо хийх үйлдэл дээр админ хэрэглэгч үүсгэх үйлдэл хамт хийгдэж байгаа учир include харилцааг дүрсэлсэн. Мөн нөөцүүд буюу хавтас ба файлуудыг жагсаан харахдаа нэрээр хайх боломжийг extend харилцаагаар дүрслэв.



Зураг 2.1: Usecase диаграм

2.3 Юзкейсийн тодорхойлолт

Дээр харуулсан юзкэйс диаграммийн тодорхойлолтуудыг доорх хүснэгтүүдэд үзүүлэв. Юзкэйсийн нэр, ID, үүсгэсэн хүний нэр, үүсгэсэн огноо, үндсэн тоглогч, нэмэлт тоглогч, товч тайлбар, өмнөх нөхцөл, үндсэн урсгал, дараах нөхцөл, альтернатив урсгал гэсэн мэдээллүүдийг оруулав.

2.3.1 Анхны тохиргоо хийх (UC-001)

Хүснэгт 2.1-д Анхны тохиргоо хийх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.1: UC-001 юзкейсийн тодорхойлолт

Юзкейс	Анхны тохиргоо хийх
ID	UC-001
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Админ
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Системийг анх ашиглах үед эхний админ хэрэглэгч болон үндсэн хадгалалтын тохиргоог үүсгэнэ.
Өмнөх нөхцөл	1. Систем дээр анхны тохиргоо хийгдээгүй байх. 2. Систем суурилагдсан бөгөөд тохиргооны дэлгэц нээгдсэн байх.
Үндсэн урсгал	1. Админ анхны тохиргооны хуудсанд хандана. 2. Систем тохиргооны талбаруудыг харуулна. 3. Админ хадгалалтын үндсэн зам, багтаамжийн хэмжээ болон admin мэдээллийг оруулна. 4. Систем мэдээллийн бүрэн бүтэн байдлыг шалгана. 5. include (Admin хэрэглэгч үүсгэх) 6. Систем үндсэн хадгалалтын бүтцийг үүсгэж тохиргоог хадгална.
Дараах нөхцөл	1. Систем анхны тохиргоо хийгдсэн төлөвт орно. 2. Анхны admin хэрэглэгч ашиглахад бэлэн болно.
Альтернатив урсгал	1. Хэрэв систем өмнө нь тохируулагдсан бол анхны тохиргооны хуудсыг цааш үргэлжлүүлэхгүй. 2. Оруулсан мэдээлэл дутуу эсвэл буруу бол систем засварлахыг сануулж дахин оруулах боломж олгоно.

2.3.2 Admin хэрэглэгч үүсгэх (UC-002)

Хүснэгт 2.2-д Admin хэрэглэгч үүсгэх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.2: UC-002 юзкейсийн тодорхойлолт

Юзкейс	Админ хэрэглэгч үүсгэх
ID	UC-002
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Админ
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Анхны тохиргооны явцад системийн эхний админ бүртгэлийг үүсгэнэ.
Өмнөх нөхцөл	1. UC-001 “Анхны тохиргоо хийх” юзкейс эхэлсэн байх. 2. Админы мэдээлэл оруулах хэсэг нээгдсэн байх.
Үндсэн урсгал	1. Админ хэрэглэгчийн нэр, и-мэйл, бүтэн нэр, нууц үгээ оруулна. 2. Систем мэдээллийн формат болон давхардлыг шалгана. 3. Систем нууц үгийг аюулгүй хэлбэрээр хадгална. 4. Систем админ эрхтэй хэрэглэгчийн бүртгэлийг үүсгэнэ.
Дараах нөхцөл	1. Анхны админ хэрэглэгч амжилттай үүссэн байна. 2. Үүссэн бүртгэл дараагийн нэвтрэлтэд ашиглагдах боломжтой болно.
Альтернатив урсгал	1. Хэрэв хэрэглэгчийн нэр эсвэл и-мэйл буруу форматтай бол систем засварлуулах хүсэлт гаргана. 2. Хэрэв нууц үгийн шаардлага хангахгүй бол бүртгэл үүсгэхгүй.

2.3.3 Хэрэглэгч бүртгэх (UC-003)

Хүснэгт 2.3-д Хэрэглэгч бүртгэх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.3: UC-003 юзкейсийн тодорхойлолт

Юзкейс	Хэрэглэгч бүртгэх
ID	UC-003
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Админ
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Админ шинэ энгийн хэрэглэгчийг системд бүртгэж, анхны хадгалалтын хязгаарыг тохируулна.
Өмнөх нөхцөл	1. Админ системд нэвтэрсэн байх. 2. Хэрэглэгч удирдах хэсэг рүү хандсан байх.
Үндсэн урсгал	1. Админ шинэ хэрэглэгч бүртгэх үйлдлийг сонгоно. 2. Хэрэглэгчийн нэр, и-мэйл, бүтэн нэр, нууц үг болон хадгалалтын хязгаарыг оруулна. 3. Систем мэдээллийн хүчинтэй эсэхийг шалгана. 4. Систем энгийн хэрэглэгчийн бүртгэлийг үүсгэнэ. 5. Систем шинэ хэрэглэгчийг жагсаалтад харуулна.
Дараах нөхцөл	1. Шинэ хэрэглэгч идэвхтэй төлөвтэйгээр бүртгэгдэнэ. 2. Хэрэглэгч системд нэвтрэх боломжтой болно.
Альтернатив урсгал	1. Хэрэв хэрэглэгчийн нэр эсвэл и-мэйл давхцвал систем бүртгэлийг үргэлжлүүлэхгүй. 2. Хэрэв хадгалалтын хязгаар буруу утгатай бол систем дахин оруулахыг шаардана.

2.3.4 Хэрэглэгчийн жагсаалт харах (UC-004)

Хүснэгт 2.4-д Хэрэглэгчийн жагсаалт харах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.4: UC-004 юзкейсийн тодорхойлолт

Юзкейс	Хэрэглэгчийн жагсаалт харах
ID	UC-004
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Админ
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Админ системд бүртгэлтэй бүх хэрэглэгчийн үндсэн мэдээлэл болон төлөвийг харна.
Өмнөх нөхцөл	1. Админ системд нэвтэрсэн байх.
Үндсэн урсгал	1. Админ хэрэглэгч удирдах хэсгийг нээнэ. 2. Систем хэрэглэгчдийн жагсаалтыг ачаална. 3. Админ хэрэглэгч бүрийн нэр, и-мэйл, үүрэг, хадгалалтын мэдээллийг шалгана.
Дараах нөхцөл	1. Хэрэглэгчийн удирдлагын мэдээлэл дэлгэц дээр харагдсан байна.
Альтернатив урсгал	1. Админ жагсаалтаас сонгосон хэрэглэгч рүү засварлах эсвэл устгах дараагийн урсгалд шилжинэ.

2.3.5 Хэрэглэгчийн мэдээлэл өөрчлөх (UC-005)

Хүснэгт 2.5-д Хэрэглэгчийн мэдээлэл өөрчлөх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.5: UC-005 юзкейсийн тодорхойлолт

Юзкейс	Хэрэглэгчийн мэдээлэл өөрчлөх
ID	UC-005
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Админ
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Админ хэрэглэгчийн үндсэн мэдээлэл болон хадгалалтын тохиргоог шинэчилнэ.
Өмнөх нөхцөл	1. Админ системд нэвтэрсэн байх. 2. Өөрчлөх хэрэглэгч бүртгэлтэй байх.
Үндсэн урсгал	1. Админ жагсаалтаас хэрэглэгчээ сонгоно. 2. Өөрчлөх мэдээллүүдийг шинэ утгаар оруулна. 3. Систем өгөгдлийн хүчинтэй эсэхийг шалгана. 4. Систем шинэ мэдээллийг хадгална.
Дараах нөхцөл	1. Хэрэглэгчийн шинэ мэдээлэл системд хадгалагдсан байна.
Альтернатив урсгал	1. Хэрэв зөвхөн хадгалалтын хязгаар өөрчлөгдөж байвал бусад талбар өмнөх утгаараа үлдэнэ. 2. Хэрэв шинэчилсэн мэдээлэл давхцвал систем өөрчлөлтийг хадгалахгүй.

2.3.6 Хэрэглэгч устгах (UC-006)

Хүснэгт 2.6-д Хэрэглэгч устгах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.6: UC-006 юзкейсийн тодорхойлолт

Юзкейс	Хэрэглэгч устгах
ID	UC-006
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Admin
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Admin сонгосон энгийн хэрэглэгчийн бүртгэлийг системээс устгана.
Өмнөх нөхцөл	1. Admin системд нэвтэрсэн байх. 2. Устгах хэрэглэгч admin биш байх.
Үндсэн урсгал	1. Admin жагсаалтаас хэрэглэгчээ сонгоно. 2. Устгах үйлдлийг баталгаажуулна. 3. Систем хэрэглэгчтэй холбоотой нэвтрэлтийн төлөв болон холбогдох өгөгдлүүдийг цэвэрлэнэ. 4. Систем хэрэглэгчийн бүртгэлийг устгана.
Дараах нөхцөл	1. Устгасан хэрэглэгч системд нэвтрэх боломжгүй болно. 2. Хэрэглэгч жагсаалтаас алга болсон байна.
Альтернатив урсгал	1. Хэрэв сонгосон хэрэглэгч олдохгүй бол систем үйлдлийг зогсооно. 2. Хэрэв admin хэрэглэгчийг сонговол систем устгахыг зөвшөөрөхгүй.

2.3.7 Системд нэвтрэх (UC-007)

Хүснэгт 2.7-д Системд нэвтрэх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.7: UC-007 юзкейсийн тодорхойлолт

Юзкейс	Системд нэвтрэх
ID	UC-007
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч нэвтрэх нэр, нууц үг ашиглан системд баталгаажиж үндсэн орчинд хандана.
Өмнөх нөхцөл	1. Хэрэглэгч системд бүртгэлтэй байх. 2. Хэрэглэгчийн бүртгэл идэвхтэй байх.
Үндсэн урсгал	1. Хэрэглэгч нэвтрэх хуудсыг нээнэ. 2. Нэвтрэх нэр болон нууц үгээ оруулна. 3. Систем оруулсан мэдээллийг шалгана. 4. Систем хэрэглэгчийг баталгаажуулж session үүсгэнэ. 5. Хэрэглэгч өөрийн үндсэн хуудас руу шилжинэ.
Дараах нөхцөл	1. Идэвхтэй session үүссэн байна. 2. Хэрэглэгч эрхтэй үйлдлүүддээ хандах боломжтой болно.
Альтернатив урсгал	1. Нууц үг буруу бол систем алдааны мэдээлэл үзүүлж дахин оролдох боломж олгоно. 2. Админ хэрэглэгч амжилттай нэвтэрвэл админд зориулсан удирдлагын боломжууд мөн нээгдэнэ.

2.3.8 Нөөцүүдийг харах (UC-008)

Хүснэгт 2.8-д Нөөцүүдийг харах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.8: UC-008 юзкейсийн тодорхойлолт

Юзкейс	Нөөцүүдийг харах
ID	UC-008
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч өөрийн болон өөртэй нь хуваалцсан файл, хавтсыг жагсаалтаар харна.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх.
Үндсэн урсгал	1. Хэрэглэгч нөөцийн үндсэн хэсгийг нээнэ. 2. Систем тухайн байршил дахь файл, хавтасны жагсаалтыг харуулна. 3. extension point: Нэрээр хайх (нэрээр хайх шаардлагатай бол) 4. Хэрэглэгч хавтас сонгож дараагийн түвшний агуулга руу шилжинэ. 5. Систем сонгосон байршлын агуулгыг шинэчилж харуулна.
Дараах нөхцөл	1. Сонгосон байршлын агуулга хэрэглэгчид харагдсан байна.
Альтернатив урсгал	1. Хэрэв хэрэглэгч нэрээр хайлт хийх шаардлагатай бол UC-009 “Нэрээр хайх” юзкейс ажиллана. 2. Хэрэглэгч өөртэй нь хуваалцсан нөөцийн хэсэг рүү шилжин тухайн эрхийн хүрээнд агуулгыг харж болно.

2.3.9 Нэрээр хайх (UC-009)

Хүснэгт 2.9-д Нэрээр хайх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.9: UC-009 юзкейсийн тодорхойлолт

Өргөтгөх юзкейс	Нэрээр хайх
ID	UC-009
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-03-12
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч өөрийн хандах эрхтэй нөөцүүдээс нэрийн дагуу хайлт хийнэ.
Сегмент 1 өмнөх нөхцөл	1. UC-008 “Нөөцүүдийг харах” юзкейсийн дэлгэц нээгдсэн байх. 2. Нэрээр хайх шаардлага үүссэн байх.
Сегмент урсгал	1. Хэрэглэгч хайлтын талбарт түлхүүр үг оруулна. 2. Систем нөөцийн нэрсээс тохирох үр дүнг хайна. 3. Систем үр дүнг жагсаалтаар харуулна. 4. Хэрэглэгч үр дүнгээс хүссэн нөөцөө сонгоно.
Сегмент 1 дараах нөхцөл	1. Хайлтын үр дүн хэрэглэгчид харагдсан байна.
Альтернатив урсгал	1. Хэрэв илэрц олдохгүй бол систем хоосон үр дүн үзүүлнэ. 2. Хэрэв хайлтын талбар хоосорвол систем ердийн жагсаалт руу буцна.

2.3.10 Шинэ хавтас үүсгэх (UC-010)

Хүснэгт 2.10-д Шинэ хавтас үүсгэх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.10: UC-010 юзкейсийн тодорхойлолт

Юзкейс	Шинэ хавтас үүсгэх
ID	UC-010
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч зөвшөөрөгдсөн байршилд шинэ хавтас үүсгэнэ.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Зорилтот байршил дээр үүсгэх эрхтэй байх.
Үндсэн урсгал	1. Хэрэглэгч хавтас үүсгэх байршлаа сонгоно. 2. Шинэ хавтас үүсгэх үйлдлийг дарна. 3. Шинэ хавтасны нэрийг оруулна. 4. Систем нэрийн хүчинтэй байдлыг шалгана. 5. Систем хавтсыг үүсгэж жагсаалтад нэмнэ.
Дараах нөхцөл	1. Шинэ хавтас сонгосон байршилд үүссэн байна.
Альтернатив урсгал	1. Хэрэв ижил нэртэй хавтас байгаа бол систем дахин нэр оруулахыг шаардана. 2. Хэрэв зорилтот байршил root түвшин бол хавтас хэрэглэгчийн үндсэн орчинд үүснэ.

2.3.11 Файл байршуулах (UC-011)

Хүснэгт 2.11-д Файл байршуулах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.11: UC-011 юзкейсийн тодорхойлолт

Юзкейс	Файл байршуулах
ID	UC-011
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч зөвшөөрөгдсөн хавтас дотор файл байршуулж хадгална.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Зорилтот хавтас дээр байршуулах эрхтэй байх.
Үндсэн урсгал	1. Хэрэглэгч файл байршуулах хавтсаа сонгоно. 2. Файлаа сонгож байршуулах үйлдлийг эхлүүлнэ. 3. Систем эрх, багтаамж, файлын хэмжээ болон нэрийг шалгана. 4. Систем файлыг хадгална. 5. Систем жагсаалтыг шинэчилж шинээр байршуулсан файлыг харуулна.
Дараах нөхцөл	1. Файл сонгосон байршилд хадгалагдсан байна. 2. Хадгалалтын ашиглалтын хэмжээ шинэчлэгдэнэ.
Альтернатив урсгал	1. Хэрэв багтаамж хүрэлцэхгүй бол систем байршуулахыг зогсооно. 2. Хэрэв дамжуулалт тасарвал хэрэглэгч дахин байршуулах боломжтой байна.

2.3.12 Нөөцийн нэр өөрчлөх (UC-012)

Хүснэгт 2.12-д Нөөцийн нэр өөрчлөх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.12: UC-012 юзкейсийн тодорхойлолт

Юзкейс	Нөөцийн нэр өөрчлөх
ID	UC-012
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч өөрийн эрхтэй файл эсвэл хавтсын нэрийг шинэчилнэ.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Сонгосон нөөц дээр нэр өөрчлөх эрхтэй байх.
Үндсэн урсгал	1. Хэрэглэгч нэр өөрчлөх нөөцөө сонгоно. 2. Шинэ нэрийг оруулна. 3. Систем нэрийн дүрэм болон давхардлыг шалгана. 4. Систем шинэ нэрийг хадгална.
Дараах нөхцөл	1. Нөөц шинэ нэрээрээ жагсаалтад харагдана.
Альтернатив урсгал	1. Хэрэв шинэ нэр хоосон эсвэл хүчингүй бол систем өөрчлөлтийг хадгалахгүй. 2. Хэрэв ижил нэртэй нөөц байвал систем дахин нэр сонгохыг шаардана.

2.3.13 Нөөц сэргээх (UC-013)

Хүснэгт 2.13-д Нөөц сэргээх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.13: UC-013 юзкейсийн тодорхойлолт

Юзкейс	Нөөц сэргээх
ID	UC-013
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-03-12
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Эзэмшигч хэрэглэгч устгасан төлөвт байгаа файл эсвэл хавтсаа сэргээнэ.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Сэргээх нөөц нь устгасан төлөвтэй байх.
Үндсэн урсгал	1. Хэрэглэгч устгасан нөөцийн хэсгийг нээнэ. 2. Сэргээх файл эсвэл хавтсаа сонгоно. 3. Сэргээх үйлдлийг баталгаажуулна. 4. Систем нөөцийг өмнөх идэвхтэй төлөвт буцаана.
Дараах нөхцөл	1. Нөөц идэвхтэй жагсаалт руу буцаж орсон байна.
Альтернатив урсгал	1. Хэрэв өмнөх байршилд ижил нэртэй нөөц үүссэн бол систем шинэ нэр сонгох эсвэл өөр байршил заахыг шаардаж болно. 2. Хэрэв хэрэглэгч сэргээхээс татгалзвал үйлдэл цуцлагдана.

2.3.14 Нөөц шилжүүлэх (UC-014)

Хүснэгт 2.14-д Нөөц шилжүүлэх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.14: UC-014 юзкейсийн тодорхойлолт

Юзкейс	Нөөц шилжүүлэх
ID	UC-014
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Эзэмшигч хэрэглэгч файл эсвэл хавтсаа өөр байршил руу шилжүүлнэ.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Шилжүүлэх нөөц болон зорилтот хавтас дээр шаардлагатай эрхтэй байх.
Үндсэн урсгал	1. Хэрэглэгч нэг буюу хэд хэдэн нөөцөө сонгоно. 2. Шилжүүлэх үйлдлийг эхлүүлнэ. 3. Зорилтот хавтсыг сонгоно. 4. Систем бүтэц болон эрхийн шалгалт хийнэ. 5. Систем нөөцийг шинэ байршилд шилжүүлнэ.
Дараах нөхцөл	1. Нөөц шинэ байршилд харагдана. 2. Хуучин байршлаас сонгосон нөөц алга болсон байна.
Альтернатив урсгал	1. Хэрэв нэг дор олон нөөц сонгосон бол систем бүгдийг хамтад нь шилжүүлнэ. 2. Хэрэв хавтсыг өөрийнх нь доторх дэд хавтас руу шилжүүлэх оролдлого гарвал систем үйлдлийг зогсооно.

2.3.15 Нөөц устгах (UC-015)

Хүснэгт 2.15-д Нөөц устгах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.15: UC-015 юзкейсийн тодорхойлолт

Юзкейс	Нөөц устгах
ID	UC-015
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Эзэмшигч хэрэглэгч өөрийн файл эсвэл хавтсыг идэвхтэй жагсаалтаас устгасан төлөв рүү шилжүүлнэ.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Сонгосон нөөц тухайн хэрэглэгчийн эзэмшлийнх байх.
Үндсэн урсгал	1. Хэрэглэгч устгах нөөцөө сонгоно. 2. Устгах үйлдлийг баталгаажуулна. 3. Систем нөөцийг идэвхтэй жагсаалтаас хасаж устгасан төлөвт шилжүүлнэ. 4. extension point: Нөөцийг бүр мөсөн устгах (сэргээх боломжгүйгээр устгахыг сонговол)
Дараах нөхцөл	1. Нөөц устгасан хэсэгт хадгалагдана. 2. Идэвхтэй жагсаалтад харагдахгүй болно.
Альтернатив урсгал	1. Хэрэв хэрэглэгч бүр мөсөн устгах шаардлагатай бол UC-016 “Нөөцийг бүр мөсөн устгах” юзкейс рүү шилжинэ. 2. Хэрэглэгч нэг дор олон нөөц сонгон устгаж болно.

2.3.16 Нөөцийг бүр мөсөн устгах (UC-016)

Хүснэгт 2.16-д Нөөцийг бүр мөсөн устгах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.16: UC-016 юзкейсийн тодорхойлолт

Өргөтгөх юзкейс	Нөөцийг бүр мөсөн устгах
ID	UC-016
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-03-12
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Устгасан төлөвт байгаа нөөцийг сэргээх боломжгүйгээр системээс бүрэн устгана.
Сегмент 1 өмнөх нөхцөл	1. UC-015 “Нөөц устгах” юзкейсийн extension point дээр ирсэн байх. 2. Нөөц устгасан төлөвт байгаа байх. 3. Хэрэглэгч сэргээх боломжгүйгээр устгахыг сонгосон байх.
Сегмент урсгал	1. Хэрэглэгч устгасан нөөцийн хэсгээс нөөцөө сонгоно. 2. Бүр мөсөн устгах үйлдлийг сонгоно. 3. Систем сэрэмжлүүлэх мэдээлэл харуулна. 4. Хэрэглэгч баталгаажуулна. 5. Систем нөөцийг физик түвшинд устгана.
Сегмент 1 дараах нөхцөл	1. Нөөц системээс бүрэн арилсан байна. 2. Тухайн нөөцийг сэргээх боломжгүй болно.
Альтернатив урсгал	1. Хэрэв хэрэглэгч баталгаажуулахгүй бол үйлдэл цуцлагдана. 2. Хэрэв нөөц олдохгүй бол систем алдааны мэдээлэл харуулна.

2.3.17 Файл татаж авах (UC-017)

Хүснэгт 2.17-д Файл татаж авах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.17: UC-017 юзкейсийн тодорхойлолт

Юзкейс	Файл татаж авах
ID	UC-017
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч өөрийн хандах эрхтэй файлыг төхөөрөмж дээрээ татаж авна.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Сонгосон файлд хандах эрхтэй байх.
Үндсэн урсгал	1. Хэрэглэгч татах файлаа сонгоно. 2. Татаж авах үйлдлийг эхлүүлнэ. 3. Систем хандалтын эрх болон файлын бэлэн байдлыг шалгана. 4. extension point: Шахсан файл хэлбэрээр татах (нэгээс олон файл эсвэл хавтас сонгосон бол) 5. Систем файлыг хэрэглэгч рүү дамжуулна.
Дараах нөхцөл	1. Файл хэрэглэгчийн төхөөрөмжид татагдсан байна.
Альтернатив урсгал	1. Хэрэв хэд хэдэн файл эсвэл хавтас хамт татах шаардлагатай бол UC-018 “Шахсан файл хэлбэрээр татах” юзкейс ашиглагдана. 2. Хэрэв дамжуулалт тасарвал хэрэглэгч дахин татах боломжтой байна.

2.3.18 Шахсан файл хэлбэрээр татах (UC-018)

Хүснэгт 2.18-д Шахсан файл хэлбэрээр татах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.18: UC-018 юзкейсийн тодорхойлолт

Өргөтгөх юзкейс	Шахсан файл хэлбэрээр татах
ID	UC-018
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-03-12
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч хэд хэдэн файл эсвэл хавтсыг нэг шахсан файл болгон татаж авна.
Сегмент 1 өмнөх нөхцөл	1. UC-017 “Файл татаж авах” юзкейсийн extension point дээр ирсэн байх. 2. Нэгээс олон файл эсвэл хавтас сонгогдсон байх. 3. Сонгосон бүх нөөцөд хандах эрхтэй байх.
Сегмент урсгал	1. Хэрэглэгч нэгээс олон файл эсвэл хавтас сонгоно. 2. Шахсан хэлбэрээр татах үйлдлийг сонгоно. 3. Систем сонгосон нөөцүүдийг түр архив файл болгон бэлтгэнэ. 4. Систем бэлтгэсэн архивыг хэрэглэгч рүү татахаар дамжуулна.
Сегмент 1 дараах нөхцөл	1. Хэрэглэгч нэг шахсан файл хүлээн авсан байна.
Альтернатив урсгал	1. Хэрэв сонголтод зөвхөн нэг файл байвал ердийн таталтын урсгал ашиглаж болно. 2. Хэрэв шахалтын явцад алдаа гарвал систем архив үүсгэхгүй бөгөөд хэрэглэгчид мэдээлнэ.

2.3.19 Нөөц хуваалцах (UC-019)

Хүснэгт 2.19-д Нөөц хуваалцах юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.19: UC-019 юзкейсийн тодорхойлолт

Юзкейс	Нөөц хуваалцах
ID	UC-019
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-02-27
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Эзэмшигч хэрэглэгч өөрийн файл эсвэл хавтсыг бусад хэрэглэгчидтэй хуваалцана.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Хуваалцах нөөц нь тухайн хэрэглэгчийн эзэмшлийнх байх.
Үндсэн урсгал	1. Хэрэглэгч хуваалцах нөөцөө сонгоно. 2. Хуваалцах үйлдлийг сонгоно. 3. Систем зорилтот хэрэглэгч сонгох боломж харуулна. 4. include (Хандалтын түвшин сонгох) 5. Систем хуваалцах мэдээллийг хадгална.
Дараах нөхцөл	1. Сонгосон хэрэглэгч тухайн нөөцөд оноосон түвшний эрхтэй болно.
Альтернатив урсгал	1. Хэрэв өмнө нь эрх олгосон хэрэглэгчийг дахин сонговол систем өмнөх эрхийн түвшинг шинэчилнэ. 2. Хэрэв зорилтот хэрэглэгч олдохгүй бол үйлдэл цуцлагдана.

2.3.20 Хандалтын түвшин сонгох (UC-020)

Хүснэгт 2.20-д Хандалтын түвшин сонгох юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.20: UC-020 юзкейсийн тодорхойлолт

Юзкейс	Хандалтын түвшин сонгох
ID	UC-020
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-03-12
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Нөөц хуваалцах үед зорилтот хэрэглэгчид олгох эрхийн түвшинг сонгоно.
Өмнөх нөхцөл	1. UC-019 “Нөөц хуваалцах” юзкейс эхэлсэн байх. 2. Хуваалцах хэрэглэгч сонгогдсон байх.
Үндсэн урсгал	1. Хэрэглэгч харах эсвэл засварлах эрхийн нэгийг сонгоно. 2. Систем сонгосон түвшинг харуулж баталгаажуулна. 3. Систем тухайн түвшний эрхийг хуваалцах мэдээлэлд хадгална.
Дараах нөхцөл	1. Зорилтот хэрэглэгчид тодорхой хандалтын түвшин оноогдсон байна.
Альтернатив урсгал	1. Хэрэв хэрэглэгч эрхийн түвшнээ өөрчлөхөөр шийдвэл баталгаажуулахын өмнө дахин сонголт хийж болно. 2. Хэрэв сонголт хийгдээгүй бол хуваалцах үйлдэл дуусахгүй.

2.3.21 Онцлох төлөв өөрчлөх (UC-021)

Хүснэгт 2.21-д Онцлох төлөв өөрчлөх юзкейсийн дэлгэрэнгүй тодорхойлолтыг үзүүлэв.

Хүснэгт 2.21: UC-021 юзкейсийн тодорхойлолт

Юзкейс	Онцлох төлөв өөрчлөх
ID	UC-021
Үүсгэсэн	Г.Алтай
Үүсгэсэн огноо	2026-03-12
Үндсэн тоглогч	Хэрэглэгч
Нэмэлт тоглогч	Байхгүй
Товч тайлбар	Хэрэглэгч өөрийн нөөцийг онцолсон төлөвт оруулах эсвэл тухайн төлөвөөс гаргана.
Өмнөх нөхцөл	1. Хэрэглэгч системд нэвтэрсэн байх. 2. Сонгосон нөөц нь тухайн хэрэглэгчийн эзэмшлийнх байх.
Үндсэн урсгал	1. Хэрэглэгч файл эсвэл хавтсаа сонгоно. 2. Онцлох төлөв өөрчлөх үйлдлийг хийнэ. 3. Систем тухайн нөөцийн онцолсон төлөвийг сольж хадгална.
Дараах нөхцөл	1. Нөөцийн онцолсон төлөв шинэчлэгдсэн байна.
Альтернатив урсгал	1. Хэрэв нөөц өмнө нь онцолсон байсан бол систем онцолсон төлөвөөс гаргана. 2. Хэрэв нөөц онцлоогүй байсан бол систем онцолсон жагсаалтад нэмнэ.

2.4 Функциональ шаардлага

1. Эхний суурилалуулалтын үе шатанд админ хэрэглэгчийн мэдээлэл, системийн ашиглах багтаамжийн зам болон хэмжээний тохируулгыг хийнэ.
2. Хэрэглэгч нэвтрэх нэр болон нууц үгээ ашиглан системд нэвтэрнэ.
3. Админ бүх хэрэглэгчийн жагсаалтыг харах боломжтой байна.
4. Админ шинэ энгийн хэрэглэгч үүсгэхдээ нэвтрэх нэр, и-мэйл, бүтэн нэр, нууц үг, хадгалалтын хязгаар зэрэг мэдээллийг оруулна.
5. Админ хүссэн хэрэглэгчийнхээ мэдээллийг шинэчлэх боломжтой байна.

6. Админ энгийн хэрэглэгчийг устгах боломжтой байна.
7. Хэрэглэгч өөрийн болон өөртэй нь хуваалцсан файл, хавтаснуудыг харах боломжтой байна. Харахдаа төрөл, нэр, засварлагдсан он сар өдрөөр ангилах боломжтой байна.
8. Хэрэглэгч устгасан файлууд, онцолсон файлууд, хуваалцсан нөөц гэсэн тусгай харагдацуудаар өгөгдлөө харах боломжтой байна.
9. Хэрэглэгч өөрийн хандах эрхтэй файл, хавтаснуудаас нэрээр хайлт хийх боломжтой.
10. Хэрэглэгч файл болон хавтсын дэлгэрэнгүй мэдээллийг харах боломжтой байна.
11. Эзэмшигч эсвэл засварлагч эрхтэй хэрэглэгч зөвшөөрөгдсөн хавтас дотор шинэ хавтас үүсгэх боломжтой байна.
12. Эзэмшигч эсвэл засварлагч эрхтэй хэрэглэгч зөвшөөрөгдсөн хавтас дотор файл байршуулах боломжтой байна.
13. Эзэмшигч эсвэл засварлагч эрхтэй хэрэглэгч зөвшөөрөгдсөн файл, хавтсын нэрийг өөрчлөх боломжтой байна.
14. Эзэмшигч хэрэглэгч өөрийн файл, хавтсуудыг өөрийн эзэмшлийн өөр хавтас руу шилжүүлэх боломжтой байна.
15. Эзэмшигч хэрэглэгч өөрийн файл, хавтсыг устгасан файлын хэсэг рүү зөөх, тэндээс сэргээх, мөн бүр мөсөн устгах боломжтой байна.
16. Хэрэглэгч өөрийн хандах эрхтэй файлыг татаж авах, урьдчилан харах, мөн хэд хэдэн файл, хавтсыг нэг дор шахсан файл хэлбэрээр татах боломжтой байна.
17. Эзэмшигч хэрэглэгч өөрийн файл, хавтас дээр бусад хэрэглэгчдэд харах эсвэл засварлах эрх олгох, эрхийн жагсаалтыг харах, мөн тухайн эрхийг цуцлах боломжтой байна.
18. Хэрэглэгч өөрийн эзэмшлийн файл, хавтсыг онцолсон төлөвт оруулах болон тухайн төлөвөөс гаргах боломжтой байна.

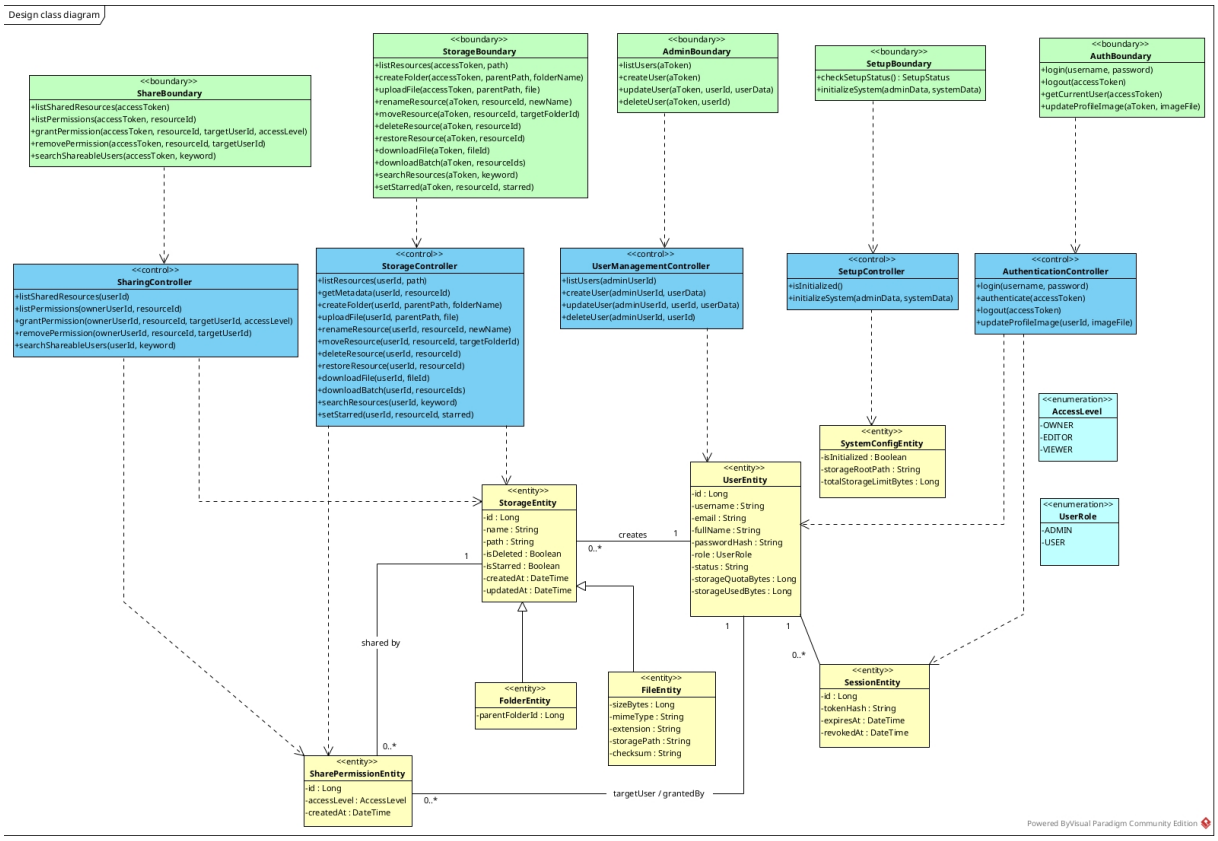
2.5 Функциональ бус шаардлага

1. Серверийн программ нь өөрийн сервер дээр байршуулан ажиллуулах боломжтой, ялангуяа Linux орчинд найдвартай ажиллахаар төлөвлөгдсөн байх.
2. Хэрэглэгчийн client аппликейшн нь web болон iOS платформ дээр ажиллах боломжтой байх.
3. Web аппликейшн нь Chrome, Firefox, Safari болон Edge браузеруудын сүүлийн хоёр хувилбар дээр зөв ажиллах боломжтой байх.
4. Хэрэглэгч болон серверийн хооронд дамжих бүх өгөгдөл аюулгүй сувгаар шифрлэгдсэн байх.

5. Хэрэглэгчийн нууц үгийг сервер дээр plaintext хэлбэрээр хадгалахгүй бөгөөд аюулгүй hash алгоритм ашиглан хадгалдаг байх.
6. Систем нь нэг сервер дээр хэд хэдэн хэрэглэгч зэрэг хандах үед үндсэн үйлдлүүдийн зөв ажиллагааг алдагдуулахгүй байх.
7. Файл байршуулах болон татах үйлдлийн явцыг хэрэглэгчийн интерфэйс дээр харуулах боломжтой байх.
8. Систем нь хэрэглэгч зөвхөн өөрийн файл болон өөрт олгогдсон эрхтэй өгөгдөлд хандах боломжийг хангах байх.
9. Файл дамжуулах болон хадгалах үед өгөгдлийн бүрэн бүтэн байдлыг шалгах боломжтой байх.

2.6 Класс диаграм

Usecase-үүдийг дараагийн шатанд дараалын диаграммаар илэрхийлэх боломжтой байлгахын тулд би энэ классын диаграммыг *boundary-control-entity* бүтэцтэйгээр боловсруулсан. Зураг 2.2-т системийн гаднах оролт хүлээн авах boundary классууд, үндсэн логикийг зохицуулах control классууд, мөн домэйн өгөгдлийг илэрхийлэх entity классуудын хамаарлыг харуулав.



Зураг 2.2: Хувийн үүлэн хадгалалтын системийн класс диаграм

Диаграммын дээд хэсэгт би *SetupBoundary*, *AuthBoundary*, *StorageBoundary*, *ShareBoundary*, *AdminBoundary* классуудыг байрлуулсан. Эдгээрийг системтэй харилцах үндсэн оролтын цэгүүд гэж үзсэн бөгөөд анхны тохиргоо, нэвтрэлт, хадгалалтын үйлдэл, хуваалцах урсгал, админы үйлдэл зэрэг usecase-үүдийн эхлэл хэсгийг төлөөлөхөөр сонгосон.

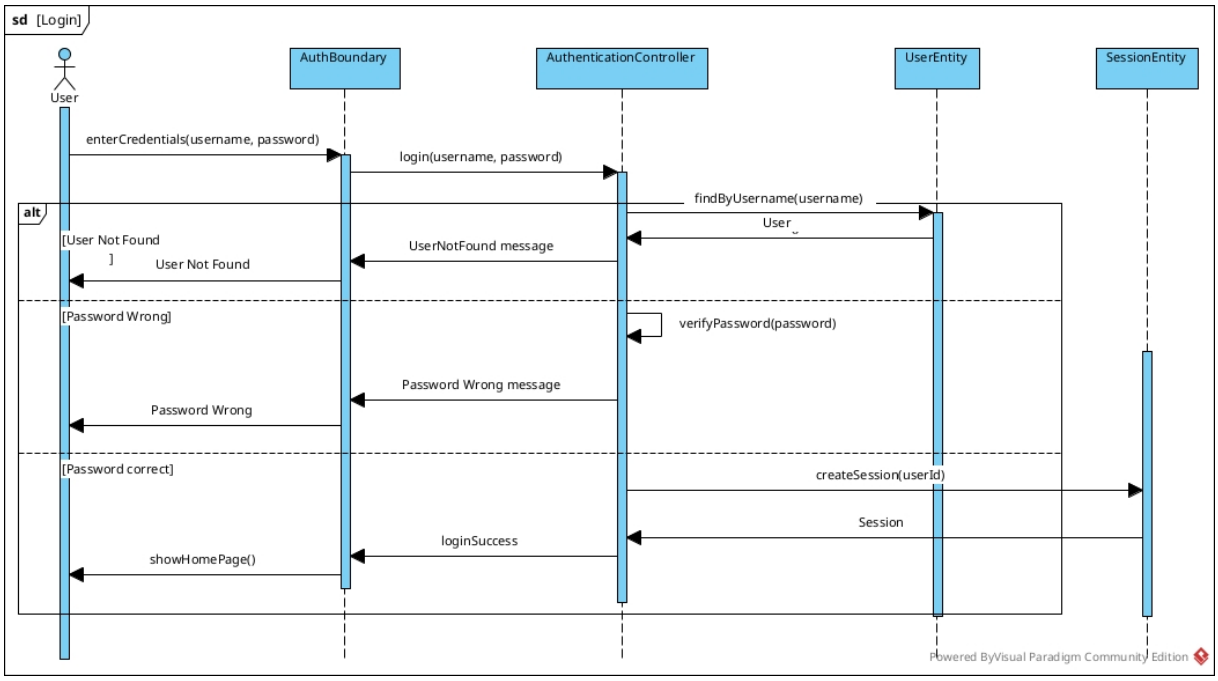
Boundary классуудын доор би *SetupController*, *AuthenticationController*, *StorageController*, *SharingController*, *UserManagementController* гэсэн control классуудыг байрлуулсан. Эдгээр нь usecase-ийн гол логикийг хариуцаж, boundary болон entity давхаргын хооронд зуучлах үүрэгтэй байхаар загварчилсан. Жишээлбэл *StorageController*-т файл, хавтастай холбоотой үндсэн урсгалуудыг, *SharingController*-т хуваалцах эрхийн логикийг, *AuthenticationController*-т нэвтрэлт ба session удирдлагыг төвлөрүүлэн оруулсан.

Entity давхаргад би *UserEntity*, *StorageEntity*, *FolderEntity*, *FileEntity*, *SessionEntity*, *SystemConfigEntity*, *SharePermissionEntity* гэсэн үндсэн классуудыг оруулсан. Энд *StorageEntity*-г хадгалалтын нийтлэг шинжийг нэгтгэсэн суурь класс байдлаар

тодорхойлж, түүнээс *FolderEntity* болон *FileEntity* өвлөгдөхөөр зохиосон. Харин *SharePermissionEntity* болон *AccessLevel* enum-аар нөөц хуваалцах эрхийн түвшинг, *UserRole* enum-аар хэрэглэгчийн үүргийг илэрхийлсэн. Иймээс энэхүү диаграммыг би системийн бүтцийг ерөнхий түвшинд харуулахын зэрэгцээ дараалын диаграмм байгуулахад оролцогч классуудыг тодорхойлох суурь болгон ашигласан.

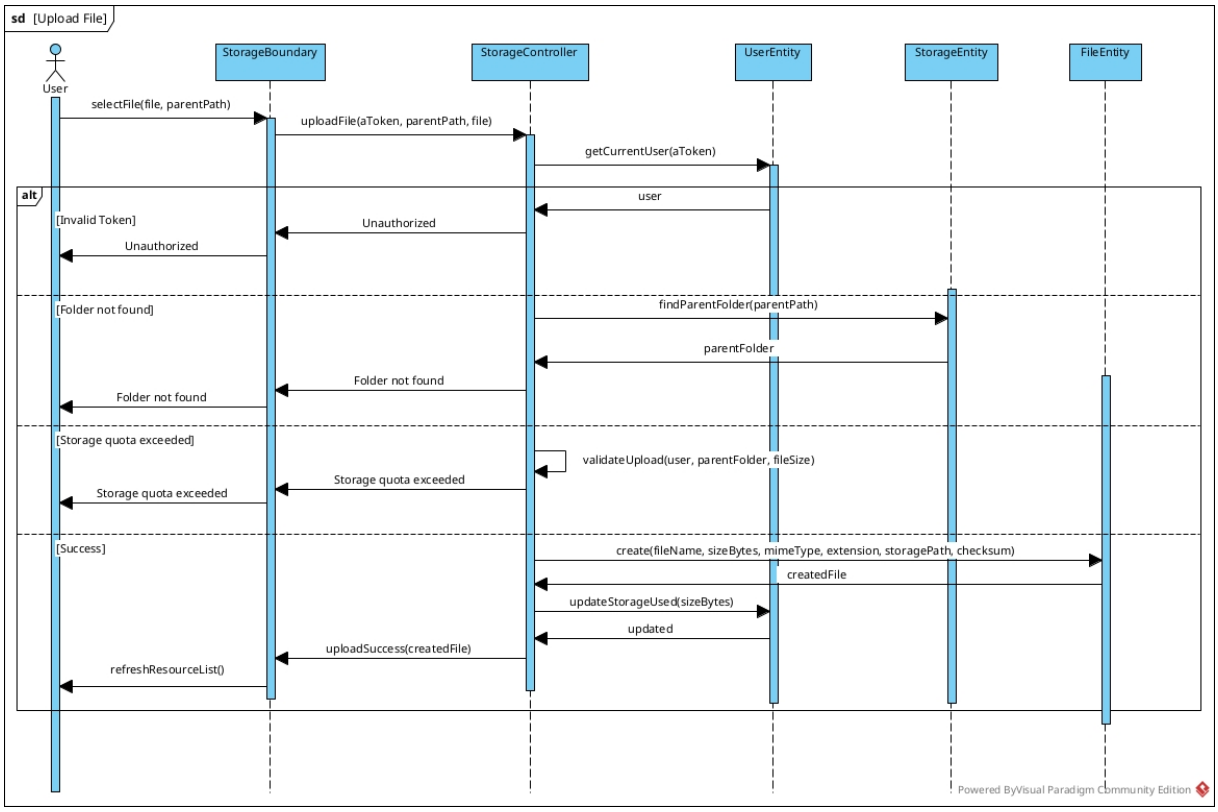
2.7 Дараалын диаграмм

Класс диаграмд тодорхойлсон boundary, control, entity классуудын харилцан ажиллагааг илүү ойлгомжтой харуулахын тулд дараах дараалын диаграммуудыг боловсруулав. Эдгээр нь системийн гол урсгал болох нэвтрэлт, файл байршуулах, нөөц хуваалцах үйл ажиллагаанд оролцох объектууд хоорондоо ямар дараалаар мэдээлэл солилцож байгааг үзүүлнэ.



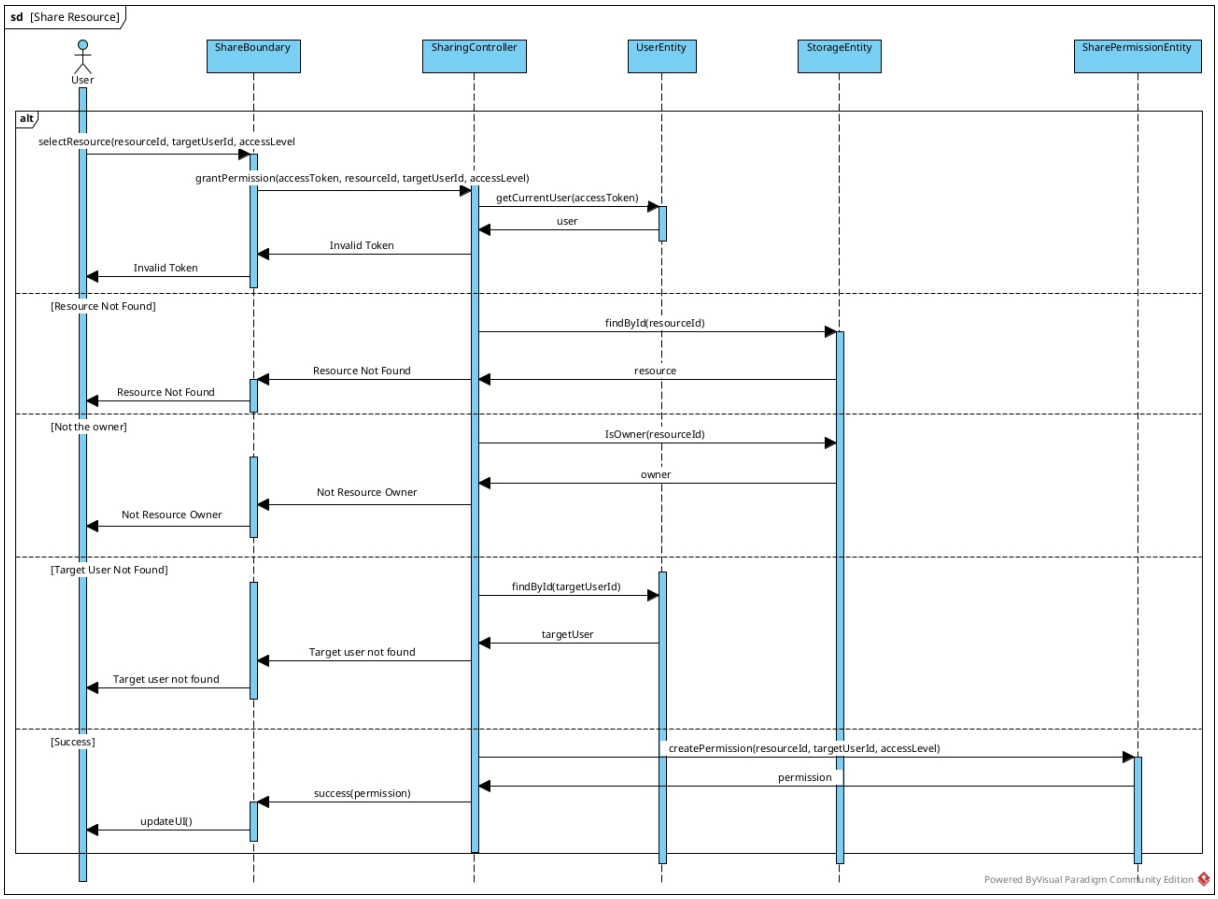
Зураг 2.3: Нэвтрэх үеийн дараалын диаграмм

Зураг 2.3-т хэрэглэгч нэвтрэх үед *AuthBoundary* хүсэлтийг хүлээн авч, *AuthenticationController* хэрэглэгчийн мэдээллийг шалган, амжилттай бол *SessionEntity* үүсгэж байгааг үзүүлсэн. Энэ дараалал нь нэвтрэлтийн үндсэн баталгаажуулалтын урсгалыг товч илэрхийлж байна.



Зураг 2.4: Файл байршуулах үеийн дараалын диаграмм

Зураг 2.4-т файл байршуулах үед *StorageBoundary* хүсэлтийг дамжуулж, *StorageController* зорилтот хавтсыг шалган, шинэ *FileEntity* болон холбогдох хадгалалтын мэдээллийг үүсгэж байгааг харуулав. Энэ нь файл байршуулах үйлдлийн гол урсгал, баталгаажуулалт болон хадгалалтын шинэчлэлтийг нэг дор үзүүлж байна.



Зураг 2.5: Нөөц хуваалцах үеийн дараалын диаграмм

Зураг 2.5-т хэрэглэгч нөөц хуваалцах үед *ShareBoundary* хүсэлтийг авч, *SharingController* зорилтот хэрэглэгч болон нөөцийг шалгасны дараа *SharePermissionEntity* үүсгэж байгааг үзүүлсэн. Ингэснээр нөөц дээр ямар хэрэглэгчид ямар түвшний эрх олгож байгааг системийн түвшинд хэрхэн бүртгэж байгааг товч тайлбарлаж байна.

2.8 Ажлын хүрээ

2.8.1 Хамрах хүрээ

Энэхүү төслийн хүрээнд дараах програм хангамжийн бүрэлдэхүүнүүдийг хөгжүүлнэ:

1. **Хувийн үүлэн хадгалалтын үндсэн серверийн программ:** Үндсэн сервер нь өөрийн сервер дээр байршиж ажиллах бөгөөд системийн анхны тохиргоо, нэвтрэлт, хэрэглэгчийн удирдлага, файл болон хавтас харах, байршуулах, татах, үүсгэх, устгах, шилжүүлэх, мөн хуваалцах эрхийн удирдлага зэрэг үндсэн боломжуудыг хангана.

2. **iOS үйлдлийн системд зориулсан мобайл клиент аппликейшн:** iOS мобайл аппликейшн нь өдөр тутмын хэрэглэгчийн үндсэн урсгалуудыг дэмжинэ. Үүнд системд нэвтрэх, өөрийн файл болон хавтсыг харах, файл байршуулах, татах, хавтас үүсгэх, өгөгдөл устгах, шилжүүлэх, өөрт хуваалцсан өгөгдөлтэй ажиллах зэрэг боломжууд орно.
3. **Веб аппликейшн:** Веб аппликейшн нь энгийн хэрэглэгчийн үндсэн урсгалуудыг дэмжихийн зэрэгцээ анхны тохиргоо хийх, хэрэглэгчийн жагсаалт харах, хэрэглэгч үүсгэх, мэдээлэл шинэчлэх, хэрэглэгч устгах зэрэг админы урсгалуудыг мөн дэмжинэ.

Бүлгийн дүгнэлт

Энэ бүлэгт хувийн үүлэн хадгалалтын системийн үндсэн шаардлагуудыг usecase, функциональ шаардлага, функциональ бус шаардлага, мөн шинжилгээний класс диаграммын түвшинд тодорхойллоо. Үүний үр дүнд хөгжүүлэх систем нь анхны тохиргоо, нэвтрэлт, хэрэглэгчийн удирдлага, файл болон хавтастай ажиллах үндсэн үйлдлүүд, хуваалцах эрхийн удирдлага зэрэг боломжуудтай байхаар төлөвлөгдөж байна. Мөн эзэмшигч, засварлагч, админ хэрэглэгчийн ялгаа, өөрийн сервер дээр ажиллах орчны хязгаарлалт, хөнгөн жинтэй байх чиглэл нь системийн зохиомжид шууд нөлөө үзүүлнэ.

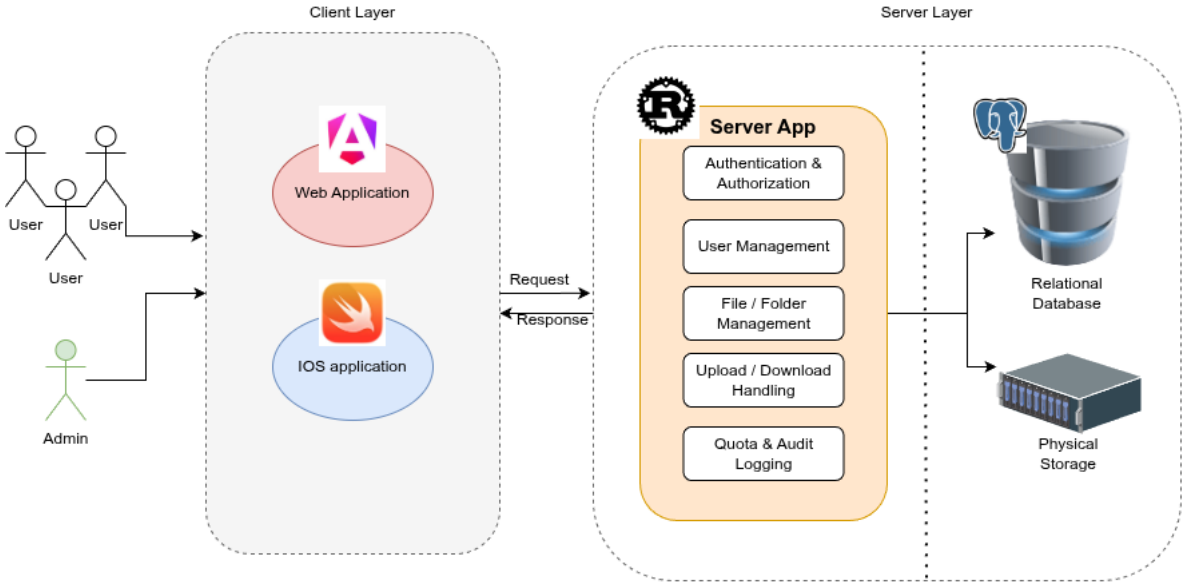
Өөрөөр хэлбэл нэгдүгээр бүлэгт хийсэн судалгааны үр дүнг энэ бүлэгт системийн шаардлага болон өгөгдлийн бүтцийн түвшинд буулгасан гэж үзэж болно. Дараагийн бүлэгт эдгээр шаардлагад тулгуурласан системийн архитектур, өгөгдлийн сангийн зохиомж, интерфэйсийн шийдлүүдийг дэлгэрэнгүй тайлбарлана.

БҮЛЭГ 3

Хувийн үүлэн хадгалалтын системийн зохиомж ба хөгжүүлэлт

3.1 Системийн архитектур

Зураг 3.1-т хувийн үүлэн хадгалалтын системийн архитектур диаграммыг харуулав. Энгийн хэрэглэгч болон admin гэсэн 2 төрлийн хэрэглэгч байх агаад бүх хэрэглэгчид веб олон IOS апп ашиглан системтэй харьцана. Веб апп-ийг Angular, IOS апп-ийг Swift технелогиудыг ашиглан хөгжүүлнэ. Тухайн хэрэглэгчийн апп-ууд нь сервертэй HTTPS холболтоор харьцаж зорилтот үйлдлүүдийг гүйцэтгэдэг байна. Сервер апп нь хэрэглэгчдийн нэвтрэлт, баталгаажуулалт, хэрэглэгчийн менежмент, файл менежмент, файл татах болон байршуулах үйлдлүүдийг хийнэ. Системийн нийт өгөгдлийг Relational өгөгдлийн бааз болон серверийн File system-д хадгална. Тухайн үүлэн хадгалалт дээр байрших файлууд болон хавтаснуудын мета өгөгдлүүд нь өгөгдлийн бааз дээр хадгалагдана.



Зураг 3.1: Системийн архитектур

3.2 Зохиомжийн үндсэн компонентууд

Системийн зохиомжид дараах үндсэн компонентуудыг тодорхойлсон.

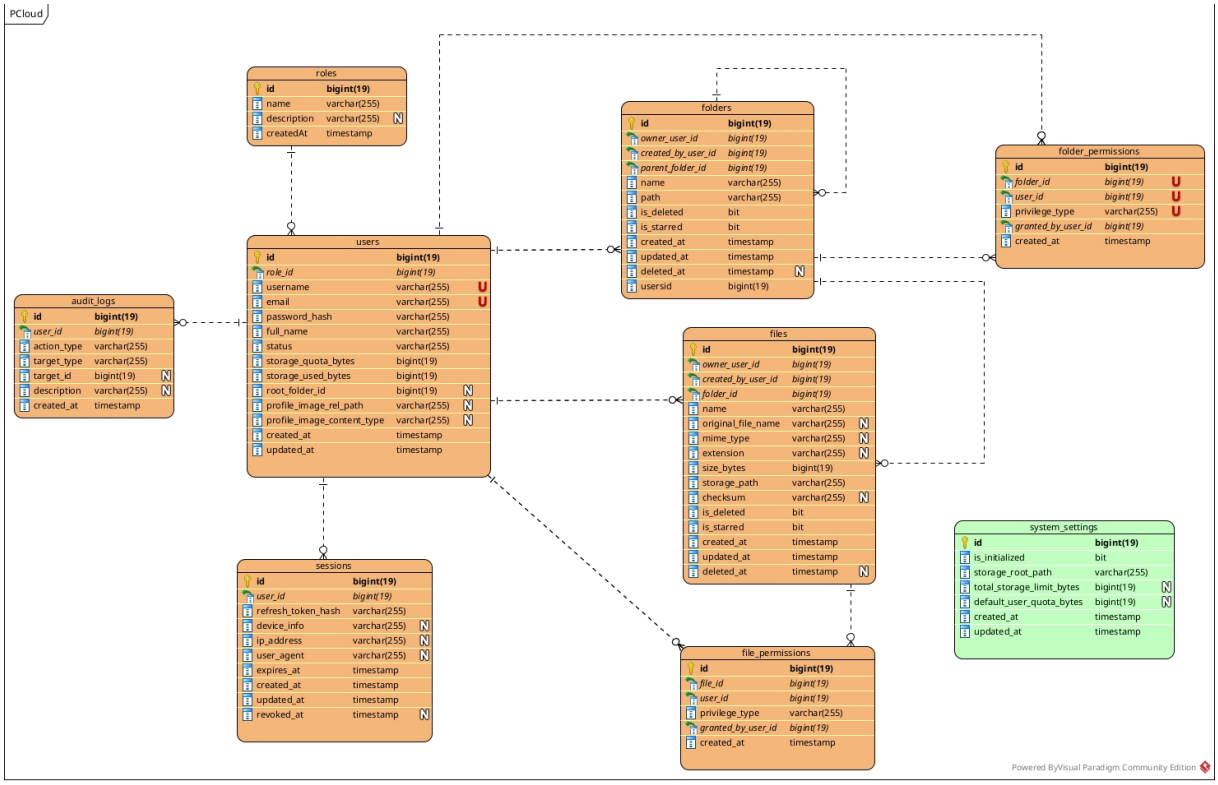
1. **Хэрэглэгчийн удирдлагын модуль** – нэвтрэх, нууц үг шалгах, session/token удирдах.
2. **Файл удирдлагын модуль** – хавтас үүсгэх, файл байршуулах, татах, нэр өөрчлөх, устгах.
3. **Хуваалцах эрхийн модуль** – хэрэглэгч болон бүлгийн түвшинд read/edit эрх олгох.

4. **Storage abstraction** – локал диск дээрх өгөгдөл болон өгөгдлийн сангийн мета мэдээллийг уялдуулах.

3.3 Өгөгдлийн ерөнхий схем

Зураг 3.2-т хувийн үүлэн хадгалалтын системийн өгөгдлийн ерөнхий схемийг харуулсан. Энэ схемд *roles* болон *users* хүснэгтүүдээр хэрэглэгч болон түүний үүргийг тодорхойлж, хадгалалтын үндсэн бүтцийг *folders* болон *files* хүснэгтүүдээр загварчилсан. *folders* хүснэгт нь *parent_folder_id* талбараар өөртэйгөө холбоо үүсгэж мод хэлбэрийн бүтэц бүрдүүлэх бол *files* хүснэгт нь тодорхой хавтаст хамааран хадгалагдана.

Мөн хуваалцах болон хандалтын эрхийн логикийг *folder_permissions* болон *file_permissions* хүснэгтүүдээр тусад нь шийдэж, файл эсвэл хавтас бүр дээр ямар хэрэглэгч ямар түвшний эрхтэй байхыг бүртгэсэн. Нэвтрэлтийн төлөв болон refresh token-тэй холбоотой мэдээллийг *sessions* хүснэгтэд, системийн үндсэн тохиргоог *system_settings* хүснэгтэд, харин хэрэглэгчийн үйлдлийн мөрийг *audit_logs* хүснэгтэд хадгалахаар төлөвлөсөн. Ингэснээр энэхүү ERD нь хадгалалт, эрхийн удирдлага, нэвтрэлт болон аудитын үндсэн өгөгдлийн холбоосуудыг нэг дор харуулж байна.

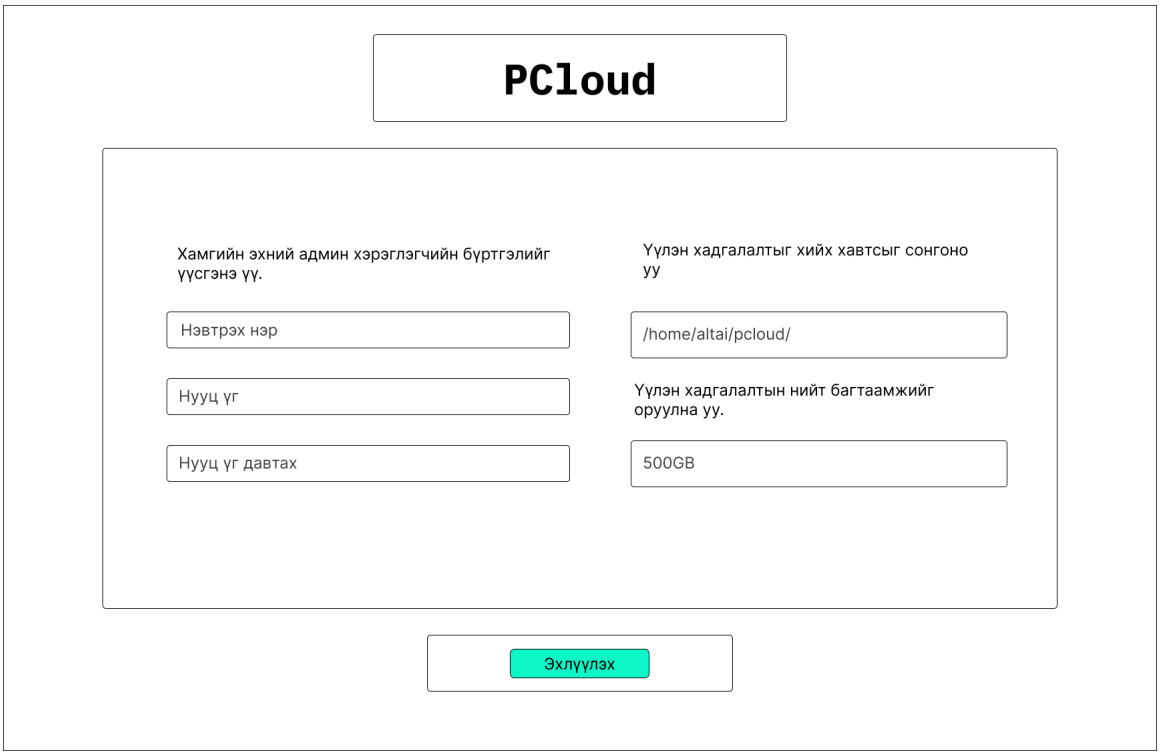


Зураг 3.2: Хувийн үүлэн хадгалалтын системийн ERD

3.4 Системийн прототип

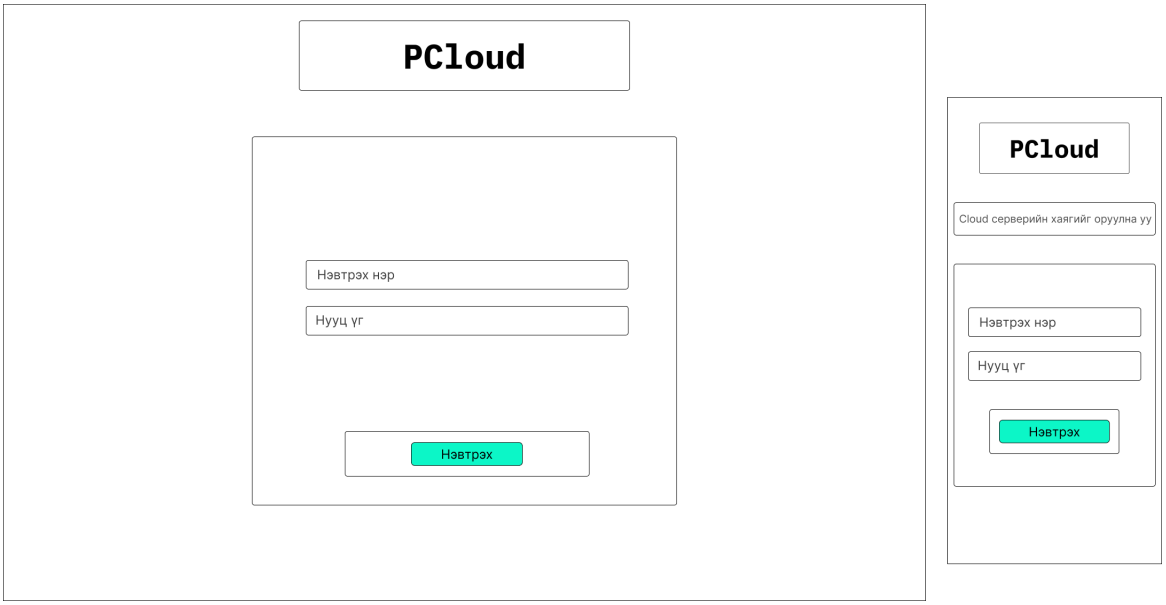
Системийн ерөнхий интерфейсийн прототипийг Figma орчинд бэлтгэсэн. Прототип нь эхний тохиргоо, нэвтрэх хуудас, энгийн хэрэглэгчийн нүүр хуудас, админ хэрэглэгчийн нүүр хуудас гэсэн үндсэн урсгалуудыг хамарсан.

Системийг анх ажиллуулах үед админ хэрэглэгч болон storage folder-ийн үндсэн тохиргоог авах дэлгэц нээгдэнэ. Энэ алхам нь self-hosted системийн хамгийн чухал эхний шат бөгөөд өгөгдөл хаана хадгалагдахыг тодорхойлно.



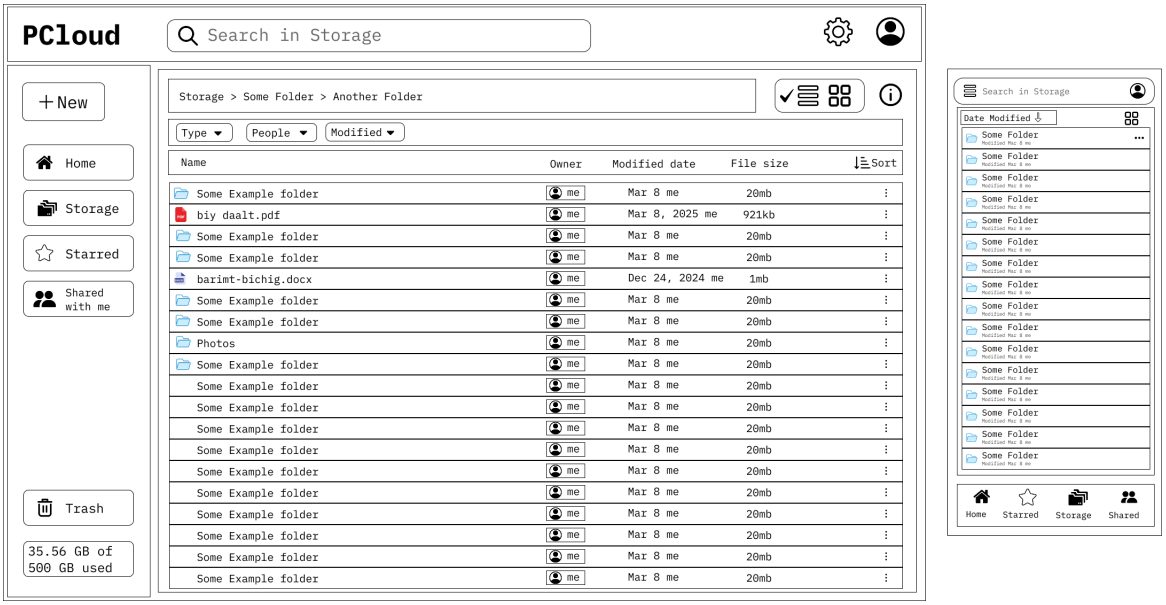
Зураг 3.3: Анхны тохиргооны дэлгэц

Нэвтрэх дэлгэц нь хэрэглэгчийн нэр, нууц үг, шаардлагатай үед серверийн хаяг оруулах талбаруудтай. Энэ загвар нь веб болон мобайл клиентээс нэг ижил backend-т холбогдох боломж олгодог.



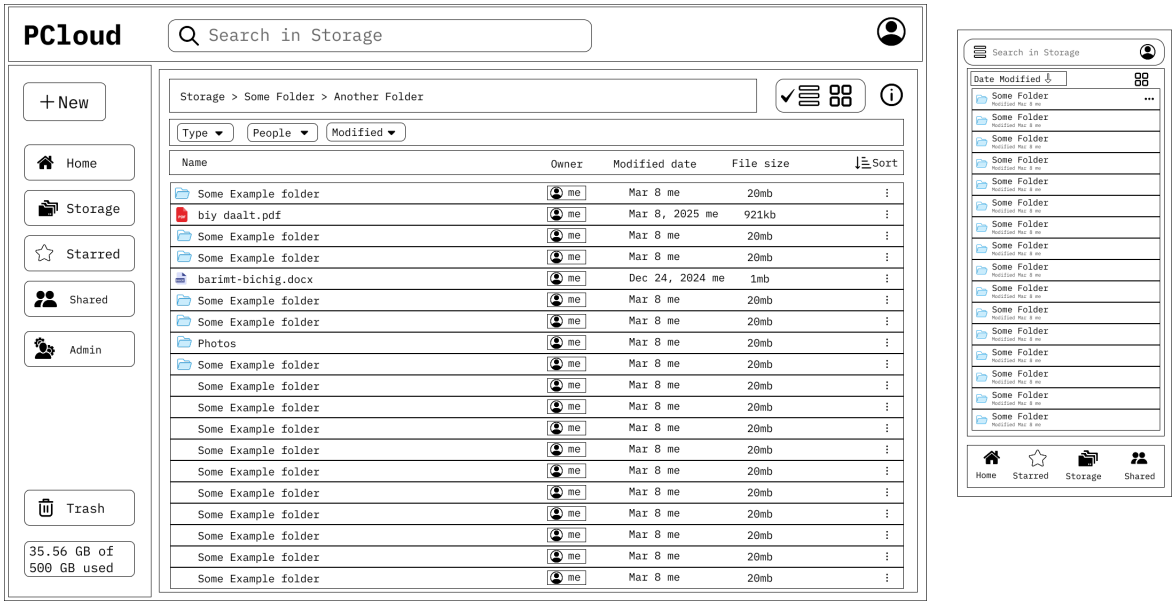
Зураг 3.4: Нэвтрэх дэлгэц

Энгийн хэрэглэгчийн нүүр хуудсан дээр файл болон хавтсын бүтэц харагдаж, upload/download, rename, delete, create folder зэрэг үндсэн үйлдлүүдийг хийх боломжтой. Мөн хайлт, харагдац өөрчлөх, shared файлууд харах зэрэг функцууд багтсан.



Зураг 3.5: Энгийн хэрэглэгчийн үндсэн хуудас

Админ хэрэглэгчийн хувьд дээрх боломжууд дээр нэмээд хэрэглэгч удирдлага, эрхийн бодлого тохируулах, системийн төлөв хянах цэсүүдийг ашиглах боломжтой.



Зураг 3.6: Админ хэрэглэгчийн нүүр хуудас

3.5 Хөгжүүлсэн системийн интерфейс

Хөгжүүлсэн интерфейс нь прототипийн үндсэн урсгалыг хадгалсан бөгөөд дараах дараалаар ажиллана.

1. Анхны тохиргоогоор админ хэрэглэгч болон хадгалалтын үндсэн хавтас үүсгэнэ (Зураг 3.3).
2. Хэрэглэгч серверийн хаяг, хэрэглэгчийн нэр, нууц үгээр нэвтэрнэ (Зураг 3.4).
3. Энгийн хэрэглэгч файлуудаа удирдаж, хуваалцсан агуулгад хандана (Зураг 3.5).
4. Админ хэрэглэгч хэрэглэгч болон системийн ерөнхий тохиргоог удирдана (Зураг 3.6).

3.6 Системд хийгдсэн тест

Системийн баталгаажуулалтыг дараах түвшинд гүйцэтгэсэн.

1. **Функциональ тест**: Нэвтрэх, файл байршуулах, татах, нэр өөрчлөх, устгах, эрх олгох урсгалуудыг usecase тус бүрээр шалгасан.
2. **Эрхийн тест**: Админ болон энгийн хэрэглэгчийн эрх ялгарах, зөвшөөрөлгүй хандалтыг блоклох шалгалт хийсэн.
3. **Интерфейсийн тест**: Веб болон мобайл клиент дээр үндсэн дэлгэцүүдийн хэрэглээний урсгал, алдааны мэдэгдлийг шалгасан.

3.7 Системийн нэвтрүүлэлт

Системийг дотоод туршилтын орчинд self-hosted хэлбэрээр нэвтрүүлж, серверийн локал storage directory-т өгөгдөл хадгалах байдлаар ажиллуулсан. Нэвтрүүлэлтийн үе шатанд хэрэглэгчийн бүртгэл үүсгэх, файл байршуулах, эрх хуваалцах, татах урсгалуудыг бодит хэрэглээний сценариар туршиж, ажиллагааны алдаануудыг засварласан.

Бүлгийн дүгнэлт

Энэ бүлэгт хувийн үүлэн хадгалалтын системийн архитектур, үндсэн компонентууд, өгөгдлийн зохиомж, прототип болон хэрэгжүүлсэн интерфейсийг тайлбарлалаа. Системийн шийдэл нь хэрэглэгчийн өгөгдлийг өөрийн сервер дээр хадгалах self-hosted зарчимд тулгуурласан бөгөөд веб болон мобайл клиентээр дамжуулан нэгдсэн API-р

ажиллахаар зохион байгуулагдсан. Мөн функцийн болон эрхийн түвшний тестүүдээр системийн үндсэн урсгалуудыг баталгаажуулсан.

Ерөнхий дүгнэлт

Энэхүү дипломын ажлын хүрээнд хэрэглэгч өөрийн төхөөрөмж дээр байршуулж ажиллуулах боломжтой хувийн үүлэн хадгалалтын системийн зохиомж, хөгжүүлэлт, туршилтыг хийж гүйцэтгэлээ. Төслийн гол зорилго нь хэрэглэгчийн өгөгдлийг гуравдагч талын серверт бус өөрийн хяналтын орчинд хадгалах, мөн веб болон мобайл клиентээр дамжуулан тухайн өгөгдөлд найдвартай хандах боломж бүрдүүлэхэд оршсон.

Судалгааны шатанд нийтийн болон self-hosted үүлэн хадгалалтын шийдлүүдийг харьцуулж, бага нөөцтэй орчинд хэрэгжих боломжтой архитектурын шаардлагыг тодорхойлсон. Шинжилгээний шатанд системийн функциональ шаардлага, хэрэглэгчийн үүрэг ба эрхийн загвар, гол usecase-уудыг боловсруулсан. Зохиомж, хөгжүүлэлтийн шатанд серверийн API, файл удирдлагын логик, эрхийн хяналт, нэвтрэлтийн урсгал, админ болон энгийн хэрэглэгчийн интерфейсийг хэрэгжүүлсэн.

Үр дүнд нь анхны тохиргоо, хэрэглэгч нэвтрэх, файл байршуулах/татах, хавтас удирдах, хуваалцах эрх олгох зэрэг үндсэн урсгалууд тогтвортой ажиллах хувийн үүлэн хадгалалтын системийг бий болгосон. Цаашид сүлжээний орчинд алсаас найдвартай холбогдох шийдлийг өргөжүүлэх, гүйцэтгэл болон аюулгүй байдлын тестийг илүү өргөн хүрээнд хийх боломжтой.

ХАВСРАЛТ А

Серверийн API-ийн жишээ

A.1 REST endpoint-ууд

Энэ хавсралтад хувийн үүлэн хадгалалтын системийн сервер талын үндсэн API endpoint-уудын жишээг үзүүлэв.

```
POST   /api/auth/login
POST   /api/auth/logout

GET    /api/users/me
POST   /api/users             // admin: create user
DELETE /api/users/{userId}    // admin: delete user

GET    /api/files?path=/      // list files/folders
POST   /api/files/upload      // upload file
GET    /api/files/download/{id} // download file
PATCH  /api/files/{id}/rename // rename file/folder
DELETE /api/files/{id}        // delete file/folder

POST   /api/shares            // grant permission
DELETE /api/shares/{shareId} // revoke permission
```

Ашигласан материалын жагсаалт

[1] Threadgold consulting. *Personal Cloud Storage Usage*. Personal Cloud Storage Usage report. 2025. URL: https://threadgoldconsulting.com/research/personal-cloud-storage-usage (**urlseen** 01/03/2026).

[2] Dean Hildebrand, Denis Serenyi. *Colossus under the hood: a peek into Google's scalable storage system*. Google's Scalable Storage Solution. 2021. URL: https://cloud.google.com/blog/products/storage-data-transfer/a-peek-behind-colossus-googles-file-system (**urlseen** 28/03/2026).

[3] Google Workspace. *Google Workspace security whitepaper: Technology with security at its core*. Google Workspace Security Whitepaper. 2026. URL: https://workspace.google.com/learn-more/security/security-whitepaper/page-4/ (**urlseen** 29/03/2026).

[4] Google Workspace Admin Help. *Choose a geographic location for your data*. Google Workspace Data Region Policy Guide. 2026. URL: https://support.google.com/a/answer/14310028?hl=en (**urlseen** 29/03/2026).

[5] Google Workspace Admin Help. *Protect your organization's data with CSE*. Google Workspace Client-side Encryption Overview. 2026. URL: https://support.google.com/a/answer/14326936?hl=en (**urlseen** 29/03/2026).

[6] Google Developers. *Google Drive API: Track changes for users and shared drives*. Google Drive API Documentation. 2026. URL: https://developers.google.com/workspace/drive/api/guides/manage-changes (**urlseen** 29/03/2026).

[7] Google Developers. *Google Drive API: Receive notifications for resource changes*. Google Drive API Push Notification Guide. 2026. URL: https://developers.google.com/workspace/drive/api/guides/push (**urlseen** 29/03/2026).

[8] Google Developers. *Google Drive API: Manage uploads*. Google Drive API Upload Guide. 2026. URL: https://developers.google.com/drive/api/v3/manage-uploads (**urlseen** 29/03/2026).

[9] Google Developers. *Google Drive API: Manage file revisions*. Google Drive API Revision Management Guide. 2026. URL: https://developers.google.com/workspace/drive/api/guides/manage-revisions (**urlseen** 29/03/2026).

[10] Google Drive Help. *Check activity and file versions*. Google Drive Version History and Activity Guide. 2026. URL: https://support.google.com/drive/answer/2409045?hl=en (**urlseen** 29/03/2026).

[11] Nextcloud. *Nextcloud Files - Open source file sync and share platform*. Official Nextcloud Files overview page. 2026. URL: https://nextcloud.com/files/ (**urlseen** 29/03/2026).

[12] Nextcloud. *Core ownCloud Contributors Fork ownCloud Into Nextcloud*. Official Nextcloud launch announcement. 2016. URL: https://nextcloud.com/blog/press_releases/pr20160602/ (**urlseen** 29/03/2026).

[13] Nextcloud GmbH. *Nextcloud Architecture Whitepaper*. Whitepaper. Accessed: 2026-03-01. Nextcloud, **july** 2018. URL: https://nextcloud.com/c/uploads/2022/03/Architecture-Whitepaper-WebVersion-072018.pdf.

[14] Nextcloud. *Configuring External Storage (GUI)*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/external_storage_configuration_gui.html (**urlseen** 29/03/2026).

[15] Nextcloud. *Files access control*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/30/admin_manual/file_workflows/access_control.html (**urlseen** 29/03/2026).

[16] Nextcloud. *Server-side Encryption*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/encryption_configuration.html (**urlseen** 29/03/2026).

[17] Nextcloud. *Controlling file versions and aging*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/file_versioning.html (**urlseen** 29/03/2026).

[18] Nextcloud. *Deleted Items (trash bin)*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_files/trashbin_configuration.html (**urlseen** 29/03/2026).

[19] Nextcloud. *Transactional file locking*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/31/admin_manual/configuration_files/files_locking_transactional.html (**urlseen** 29/03/2026).

[20] Nextcloud. *Memory caching*. Nextcloud Administration Manual. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/configuration_server/caching_configuration.html (**urlseen** 29/03/2026).

[21] Nextcloud. *Nextcloud System Requirements*. NextCloud official Documentation. 2026. URL: https://docs.nextcloud.com/server/stable/admin_manual/installation/system_requirements.html (**urlseen** 28/03/2026).

[22] ownCloud. *Scalable, secure, faster and more efficient: ownCloud Infinite Scale is here*. Official ownCloud Infinite Scale release announcement. 2022. URL: https://owncloud.com/news/infinitescalerelease/ (**urlseen** 29/03/2026).

[23] ownCloud. *Architecture and Concepts of Infinite Scale*. ownCloud Infinite Scale Architecture Documentation. 2026. URL: https://doc.owncloud.com/ocis/next/architecture/architecture.html (**urlseen** 29/03/2026).

[24] ownCloud. *Security Aspects of Infinite Scale*. ownCloud Infinite Scale Security Documentation. 2026. URL: https://doc.owncloud.com/ocis/next/security/security.html (**urlseen** 29/03/2026).

[25] ownCloud. *Availability and Scalability*. ownCloud Infinite Scale Availability and Scaling Guide. 2026. URL: https://doc.owncloud.com/ocis/next/availability_scaling/availability_scaling.html (**urlseen** 29/03/2026).

[26] Rust. *The Rust Programming Language*. Book about Rust. 2026. URL: https://doc.rust-lang.org/book/ (**urlseen** 02/03/2026).

[27] The PostgreSQL Global Development Group. *About PostgreSQL*. Official PostgreSQL overview page. 2026. URL: https://www.postgresql.org/about/ (**urlseen** 31/03/2026).